ESSENTIALS OF WEB PROGRAMMING
UNIT-III

3. Advanced JavaScript, XML & AJAX

Introduction

Modern websites are expected to do much more than display static text and images. A user may log in, submit a form, search for information, add products to a cart, receive data from a server, and see part of a webpage update without reloading the entire page. These requirements led to the development and wider use of **advanced JavaScript features, XML, and AJAX.**

Advanced JavaScript provides structured and asynchronous programming techniques for developing interactive applications. XML provides a standard text-based format for representing and exchanging structured data. AJAX combines JavaScript with browser communication techniques so that a webpage can exchange data with a server in the background and update selected parts of the page dynamically.

Advanced JavaScript is therefore needed for:

- organizing programs using classes and modules;
- reusing functionality through inheritance;
- performing operations without blocking the webpage;
- handling delayed operations using callbacks and promises;
- communicating with servers using APIs;
- validating forms and updating webpages dynamically.

Scenario

Consider an online shopping website. When customer clicks **Place Order**, the webpage may need to contact the server, verify stock, process payment

information, and display the result. Some of these operations require time to complete. Advanced JavaScript allows the website to perform such operations without freezing the webpage while the user waits.

Scenario

Consider a **college student-result portal.**

Student enters Register Number
↓
JavaScript validates the input
↓
Request sent to the server
↓
Server searches for student result
↓
Result returned as XML or JSON
↓
JavaScript receives the response
↓
Result displayed without reloading the page

Need of this Unit:

The concepts in this unit are required because modern web applications depend heavily on structured programming and communication between the browser and server.

understands only basic HTML, CSS, and JavaScript can create interactive webpages, but developing more realistic web applications requires knowledge of:

1. **Classes and inheritance** to organize and reuse program logic.
2. **Modules** to divide a large JavaScript application into manageable files.

3. **Callbacks, promises, and async/await** to handle operations that take time.
4. **Fetch API and XMLHttpRequest** to communicate with servers.
5. **XML** to understand structured data representation.
6. **AJAX** to update webpages without complete page reloads.
7. **JSON response handling** to process the data format commonly returned by modern web services.

3.1 Classes, Inheritance and Modules

3.1.1 Introduction

In basic JavaScript programs, variables, arrays, objects, and functions are sufficient for performing simple operations. However, real websites may contain large amounts of related data and many operations. For example, an e-commerce website may contain hundreds of products, and every product may have a name, price, category, stock quantity, and functions for displaying or updating its details.

If all these details are written separately for every product, the JavaScript program becomes lengthy and difficult to maintain. JavaScript therefore provides **classes** for organizing similar objects, **inheritance** for reusing existing class features, and **modules** for dividing a large program into smaller reusable files.

These concepts are especially useful when developing larger and more organized web applications.

3.1.2 Need for Classes, Inheritance and Modules

Consider an online shopping website containing 500 products. Every product has common information such as:

- Product name
- Price
- Stock quantity
- Category

Without classes, a developer may create separate objects repeatedly:

```
const product1 = {
        name: "Laptop",
        price: 50000
};
```

```
const product2 = {
        name: "Mobile",
        price: 20000
};
```

This approach is manageable for two products, but becomes difficult when many similar objects are required.

A **class** provides one common structure from which many product objects can be created.
Similarly, suppose the website sells normal products and digital products. Both contain a name and price, while digital products additionally contain a download size. **Inheritance** allows the digital-product class to reuse the common features instead of rewriting them.

Finally, a large website may contain thousands of lines of JavaScript. **Modules** allow the program to be divided into files such as:

product.js
cart.js
login.js
payment.js

Thus, classes improve organization, inheritance improves code reuse, and modules improve program maintenance.

3.1.3 JavaScript Class

Definition: A class in JavaScript is a programming construct that defines a common structure for creating objects with related properties and methods. It specifies the information that objects can store and the operations that can be performed on that information. Multiple objects

can be created from the same class, while each object can maintain its own values. Classes therefore provide an organized and reusable approach for representing similar entities in a JavaScript application.

Syntax

```
class ClassName {
      // Properties and methods
}
```

Example

```
class Product {
}
```

Here, Product is the name of the class.

Scenario

An online shopping website contains many products. Instead of designing every product separately, the developer can create one Product class and use it to create Laptop, Mobile, Keyboard, and other product objects.

3.1.4 Object

Definition: An **object is an individual instance created from a class and represents a particular entity in a program.** It can contain its own values for the properties defined by the class and can use the methods associated with that class. Different objects created from the same class share a common structure but may contain different data. Objects are commonly used to represent entities such as students, products, customers, books, and employees.

Example

```
class Product {
}
```

```
const product1 = new Product();
```

Here:

Product → Class

product1 → Object

Scenario

If Product is a class, individual objects may represent:

Laptop

Mobile Phone

Keyboard

Headphones

Each is a different product, but all follow the structure of the Product class.

3.1.5 The new Keyword

Definition: The **new keyword is used to create a new object from a JavaScript class or constructor.** When new is used with a class, JavaScript creates a fresh object and automatically executes the class constructor. The newly created object can then store its own property values and use the methods defined by the class.

Syntax

```
const objectName = new ClassName();
```

Example

```
const product1 = new Product();
```

Scenario

Whenever a new product is added to an e-commerce website, the program can create a new product object using the new keyword.

3.1.6 Constructor

Definition: A constructor is a special method of a JavaScript class that is automatically executed when a new object is created. It is mainly used to initialize the properties of the newly created object with suitable starting values. Values supplied during object creation can be received by the constructor as parameters. A JavaScript class can contain only one method named constructor.

Syntax

```
class Product {

    constructor(name, price) {
        this.name = name;
        this.price = price;
    }
}
```

Creating an Object

```
const product1 =
    new Product("Laptop", 50000);
```

When this object is created:

name = Laptop

price = 50000 are passed to the constructor.

Scenario

When an administrator adds a product to an online store, the product name and price can be supplied immediately while creating the product object.

3.1.7 Properties

Definition: A property is a named value associated with an object and represents a characteristic or state of that object. Properties store information such as a product's name, a student's roll number, or a customer's account balance. Objects created from the same class may have the same property names but different property values.

Example

```
class Product {

        constructor(name, price) {
                this.name = name;
                this.price = price;
        }
}
```

Here:

this.name

this.price

represent object properties.

Scenario

For an e-commerce product, useful properties may include:

name

price

stock

category

A Laptop and a Mobile may have different values for these properties.

3.1.8 Methods

Definition: A method is a function defined as part of a class or object to perform an operation related to that object. Methods describe the behaviour of objects and can access their properties when performing calculations or producing results. By placing related operations inside a class, JavaScript keeps data and the functions that work with that data together.

Example

```
class Product {

        constructor(name, price) {
                this.name = name;
                this.price = price;
        }

        displayProduct() {
                return this.name + " - ₹" + this.price;
        }
}
```

Calling the method:

```
const product1 =
        new Product("Laptop", 50000);

console.log(product1.displayProduct());
```

Output:

```
Laptop - ₹50000
```

Scenario

A shopping website can use a displayProduct() method whenever product details need to be shown to a customer.

3.1.9 This Keyword

Definition: This keyword in JavaScript refers to the object associated with the current execution context. Within a class constructor or instance method, it is commonly used to access the properties and methods of the current object. This allows the same class code to operate independently on many different objects.

Example

```
class Student {

        constructor(name) {
                this.name = name;
        }
}
Creating two objects:
const student1 =
        new Student("Ananya");

const student2 =
        new Student("Rahul");
```

Here:

student1.name → Ananya

student2.name → Rahul

Scenario

A college portal may create hundreds of student objects. this.name ensures that each student object's name is stored and accessed separately.

3.1.10 Inheritance

Definition: Inheritance is an object-oriented programming mechanism through which one class can acquire and reuse the properties and methods of another class. The existing class provides common functionality, while the derived

class can reuse that functionality and add more specialized features. Inheritance reduces repeated code and helps represent relationships between general and specialized entities.

Simple Example

Suppose an online store contains:

Product

↓

DigitalProduct

Every digital product has:

Name

Price just like a normal product.

Scenario

An online bookstore sells both printed books and e-books. An e-book is still a product, so it can inherit common product details such as name and price and add digital information such as file size.

3.1.11 Parent Class and Child Class

Parent Class – Definition: A parent class is an existing class whose properties and methods can be inherited by another class. It generally contains common features that are applicable to several related classes. The parent class therefore provides a general structure that can be reused by more specialized classes.

Example:

class Product { }

Here, Product can act as the parent class.

Child Class – Definition: A child class is a class that inherits properties and methods from another class and may also define additional features

of its own. It represents a more specialized form of the parent class. In JavaScript, the inheritance relationship is established using the extends keyword.

Example:

```
class DigitalProduct extends Product {
}
```

Scenario

Parent Class → Product

Child Class → DigitalProduct

The child class inherits common product features but may add a property such as fileSize.

3.1.12 The extends Keyword

Definition: The **extends keyword is used in a JavaScript class declaration to create an inheritance relationship between a child class and a parent class.** It allows the child class to access and reuse methods available through the parent class. The child class may also define additional properties and methods specific to its own purpose.

Syntax

```
class ChildClass extends ParentClass {
}
```

Example

```
class Product {

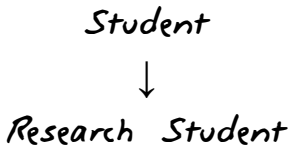
        displayMessage() {
                return "Product Available";
        }
}
```

```
class DigitalProduct extends Product {
}
```

Now an object of DigitalProduct can use:
displayMessage() even though the method was written inside Product.

Scenario

A college portal may contain:



Research Student can inherit common student operations such as displaying the student's name and department.

3.1.13 The super () Keyword

Definition: The **super () function is used within the constructor of a child class to invoke the constructor of its parent class.** It enables the child object to initialize the properties that are defined by the parent class. In a derived class constructor, super () must be called before accessing this.

Example

```
class Product {

    constructor(name, price) {
        this.name = name;
        this.price = price;
    }
}

class DigitalProduct extends Product {
```

```
        constructor(name, price, fileSize) {

            super(name, price);

            this.fileSize = fileSize;
        }
}
```

The statement:
`super(name, price);` sends the common values to the parent constructor.

Scenario
An e-book needs a name and price, just like any other product. Instead of repeating the initialization code, the child class sends these values to the Product constructor using super().

3.1.14 Complete Program - Class and Inheritance

```
<!DOCTYPE html>
<html>

<head>
    <title>Online Book Store</title>
</head>

<body>

<h2>Digital Product Details</h2>

<p id="result"></p>

<script>
```

```
class Product {

    constructor(name, price) {
        this.name = name;
        this.price = price;
    }

    displayProduct() {
        return "Product: " + this.name +
               "<br>Price: ₹" + this.price;
    }
}


class DigitalProduct extends Product {

    constructor(name, price, fileSize) {

        super(name, price);

        this.fileSize = fileSize;
    }

    displayDigitalProduct() {

        return this.displayProduct() +
               "<br>File Size: " +
               this.fileSize;
    }
}


const book =
```

```
        new DigitalProduct(
            "Web Programming eBook",
            500,
            "25 MB"
        );


document.getElementById("result").innerHTML =
        book.displayDigitalProduct();

</script>

</body>

</html>
```

3.1.15 JavaScript Modules

Definition: A JavaScript module is a separate JavaScript file that contains reusable variables, functions, classes, or other program components that can be shared with other modules. Modules allow large applications to be divided into smaller and logically organized files. They reduce unnecessary duplication and make programs easier to understand, test, reuse, and maintain.

Why Are Modules Required?

Consider an e-commerce application containing:

Login
Products
Cart
Orders
Payment

If all JavaScript code is written in one file:

script.js

the file may become very large.

Instead, the program can be divided into:

login.js

product.js

cart.js

order.js

payment.js

Each module performs a specific responsibility.

Scenario

An online store may keep product-related code in product.js and shopping-cart code in cart.js. This allows developers to modify one part of the application without unnecessarily changing the other parts.

3.1.16 The export Statement

Definition: The **export statement makes selected variables, functions, classes, or other declarations of a JavaScript module available for use in other modules.** It allows functionality to be written once and reused wherever required. JavaScript supports both named exports and default exports, although named exports are sufficient for a beginner introduction.

Example – product.js

```
export class Product {

    constructor(name, price) {
        this.name = name;
        this.price = price;
    }
}
```

The Keyword: export

makes the Product class available to another JavaScript module.

Scenario

An e-commerce website can define the Product class once in product.js and export it so that different parts of the website can reuse the same class.

3.1.17 The import Statement

Definition: The **import statement is used to access functionality that has been exported from another JavaScript module.** It enables the current module to use variables, functions, or classes without rewriting their definitions. Imports therefore support code reuse and separation of program responsibilities.

Example - app.js

```
import { Product } from "./product.js";

const product1 =
      new Product("Laptop", 50000);

console.log(product1.name);
```

Output: Laptop

Scenario

The main shopping page can import the Product class from product.js and use it to create product objects.

3.1.18 Using Modules in HTML

A JavaScript module is connected to an HTML page using:
<script type="module" src="app.js"></script>

Definition: The **type="module" attribute instructs the browser to treat the referenced JavaScript file as an ECMAScript module.** This enables the file to use module features such as import and export. Module scripts also follow browser loading and execution rules designed specifically for JavaScript modules.

Simple Module Structure

online-store/

index.html
product.js
app.js

product.js

```
export class Product {

    constructor(name, price) {
        this.name = name;
        this.price = price;
    }

    displayProduct() {
        return this.name + " - ₹" + this.price;
    }
}
```

app.js

```
import { Product } from "./product.js";

const product1 =
      new Product("Laptop", 50000);

document.getElementById("result").textContent =
      product1.displayProduct();
```

index.html

```
<!DOCTYPE html>
<html>

<head>
      <title>Online Store</title>
</head>

<body>

<h2>Product Details</h2>

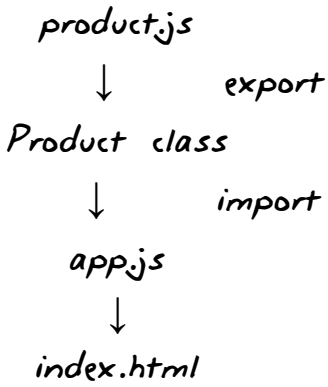
<p id="result"></p>

<script type="module" src="app.js"></script>

</body>

</html>
```

How It Works



product.js defines reusable functionality, **app.js** imports and uses it, **index.html** loads the main module.

3.1.19 Class, Inheritance and Modules – Comparison

Concept	Purpose	Website Example
Class	Defines a common structure	Product
Object	Represents one actual entity	Laptop
Property	Stores object information	Price
Method	Performs an object operation	Display product
Constructor	Initializes a new object	Set name and price
this	Refers to the current object	Current product
Inheritance	Reuses features of another class	DigitalProduct inherits Product
extends	Establishes inheritance	DigitalProduct extends Product
super()	Calls parent constructor	Initialize name and price
Module	Organizes code into separate files	product.js
export	Shares module functionality	Export Product
import	Uses shared functionality	Import Product

3.1.20 Scenario-Based Questions

Question 1: An online store contains 500 products, and every product has a name, price, and stock value. Which concept can define their common structure?
Answer: A JavaScript class.

Question 2: A Laptop is created from the Product class. What does the Laptop represent?
Answer: An object of the Product class.

Question 3: A product must receive its name and price immediately when it is created. Which class feature should be used?
Answer: A constructor.

Question 4: Inside a class, the developer needs to access the current product's price. Which keyword is commonly used?
Answer: this.

Question 5: An e-book should reuse the name and price features of a normal product but also store its file size. Which concept should be used?
Answer: Inheritance.

Question 6: Which keyword establishes inheritance between two JavaScript classes?
Answer: extends.

Question 7: Which statement calls the parent class constructor from a child class?
Answer: super().

Question 8: An e-commerce website stores product, cart, and payment functions in different JavaScript files. Which concept is being applied?
Answer: JavaScript modules.

3.1.21 Important Questions

1. What is a JavaScript class?
A JavaScript class defines a common structure for creating objects containing related properties and methods. It enables multiple similar objects to be created using one reusable definition.

2. What is an object?
An object is an individual instance created from a class. It contains its own values while following the structure defined by the class.

3. What is a constructor?
A constructor is a special class method that automatically executes when a new object is created and is mainly used to initialize object properties.

4. What is the purpose of this?
Within a class constructor or instance method, this commonly refers to the current object and is used to access its properties and methods.

5. What is inheritance?
Inheritance allows one class to acquire and reuse features from another class. It reduces repeated code and supports specialized child classes.

6. What is the difference between a parent class and a child class?
A parent class provides common features, while a child class inherits those features and may add its own specialized functionality.

7. What is the purpose of super()?
super() invokes the parent class constructor from a child class constructor.

8. What is a JavaScript module?

A JavaScript module is a separate JavaScript file containing reusable program components that can be shared with other modules.

9. What is the difference between export and import?

export makes module functionality available to other files, while import brings that exported functionality into another module.

3.1.22 Summary

A **class** provides a reusable structure for creating similar objects. Objects contain properties representing their data and methods representing their behaviour. The constructor initializes new objects, while this is commonly used within class code to refer to the current object.

Inheritance allows a child class to reuse and extend the functionality of a parent class. JavaScript uses extends to establish inheritance and super() to call the parent constructor.

JavaScript modules divide large applications into smaller reusable files. The export statement shares code from one module, while import makes that code available in another module. Together, classes, inheritance, and modules help developers create organized, reusable, and maintainable JavaScript applications.

3.2 Asynchronous Programming, Callbacks and Promises

3.2.1 Introduction

JavaScript programs often perform operations that do not produce an immediate result. For example, a website may request student details from a server, wait for an online payment response, load an image, or obtain product information from a database. Some of these operations may require a few milliseconds or several seconds to complete.

If JavaScript stopped the entire webpage until every such operation was completed, the website could become slow and unresponsive. **Asynchronous programming** allows time-consuming operations to proceed without unnecessarily blocking the remaining program.
To understand asynchronous programming, beginners should first understand the difference between **synchronous** and **asynchronous** execution.

3.2.2 Synchronous Programming

Definition: Synchronous programming is a method of program execution in which statements are executed in sequence, and each operation is completed before the next operation proceeds. The execution order normally follows the order in which the statements appear in the program. If one operation takes a long time to complete, subsequent operations may have to wait. Synchronous execution is simple to understand and is suitable for operations that produce results immediately.

Simple Example

```
console.log("Step 1");
console.log("Step 2");
console.log("Step 3");
```

Output:

Step 1

Step 2

Step 3

JavaScript executes the statements one after another.

Scenario

Consider a college webpage that performs a simple marks calculation.

```
let mark1 = 80;
let mark2 = 90;

let total = mark1 + mark2;

console.log(total);
```

The calculation is completed immediately, so normal synchronous execution is sufficient.

3.2.3 Problem with Synchronous Execution

Some website operations require time to complete.

For example, a college result portal may send a student register number to a server and wait for the result. The server may require one or two seconds to process the request.

Conceptually:

Send Request

↓

Wait for Server

↓

Receive Result

↓

Continue Program

If the entire webpage had to stop during this waiting period, the user might not be able to interact smoothly with the page.

This creates the need for asynchronous programming.

3.2.4 Asynchronous Programming

Definition: Asynchronous programming is a programming approach in which an operation can be initiated without requiring the rest of the program to remain blocked until that operation is completed. The program can continue performing other work while waiting for the asynchronous operation to finish. When the operation completes, JavaScript processes its result using mechanisms such as callbacks, promises, or async/await.

Asynchronous programming is particularly important in web applications that communicate with servers or handle other delayed operations.

Example

```
console.log("Start");

setTimeout(function () {
        console.log("Data received");
}, 2000);

console.log("Continue");
```

Output:

Start

Continue

Data received

The message "Data received" appears after approximately two seconds.

The program does not wait at the setTimeout() statement. It continues and displays "Continue".

Scenario

Suppose an e-commerce website is loading product recommendations. The user should still be able to view the existing page while the recommended products are being prepared. Asynchronous programming allows the recommendation process to complete later without stopping normal interaction with the page.

3.2.5 setTimeout() for Understanding Asynchronous Behaviour

Definition: setTimeout() is a browser function that schedules a function to be executed after a specified minimum delay. The delay is given in milliseconds, where 1000 milliseconds represent approximately one second. It is commonly used in beginner examples to demonstrate delayed and asynchronous behaviour. It does not pause the entire JavaScript program while the delay is occurring.

Syntax

```
setTimeout(function, delay);
```

Example

```
setTimeout(function () {
        console.log("Welcome Student");
}, 2000);
```

The function is scheduled to execute after approximately two seconds.

Scenario

A shopping website may display a message a short time after the user completes an order:

```
setTimeout(function () {
```

```
    console.log("Thank you for your order");
}, 2000);
```

This is only a simple demonstration of delayed execution. Real server communication will be studied later using the Fetch API and AJAX.

3.2.6 Callback Function

Definition: A callback is a function that is passed to another function so that it can be executed at an appropriate time, often after an operation has completed. Callbacks allow one part of a program to specify what should happen after another task finishes. They are widely used in event handling and asynchronous programming. A callback may be written as a named function or as an anonymous function.

Example

```
function displayMessage() {
    console.log("Task Completed");
}

setTimeout(displayMessage, 2000);
```

Here:
displayMessage is passed to setTimeout() as a callback function.

After the delay, JavaScript calls the function.

3.2.7 Callback with a Website Scenario

Consider a college website that is temporarily simulating the process of retrieving a student result.

```
function showResult() {
```

```
    console.log("Student Result: Pass");
}

console.log("Searching for result...");

setTimeout(showResult, 2000);
```

Output:

Searching for result...

Student Result: Pass

The showResult() function does not execute immediately. It is called after the delayed operation completes.

How It Works

First:

```
console.log("Searching for result...");
```

displays the initial message.

Then:

```
setTimeout(showResult, 2000);
```

schedules the callback.

After approximately two seconds:

```
showResult();
```

is executed automatically.

3.2.8 Callback with Data

A callback can also receive information.

Example

```
function displayProduct(productName) {
        console.log("Product: " + productName);
}

function loadProduct(callback) {

        setTimeout(function () {
                callback("Laptop");
        }, 2000);
}

loadProduct(displayProduct);
```

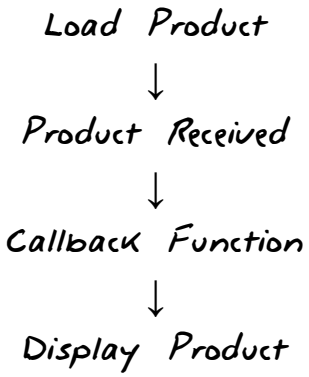
Output:

```
Product: Laptop
```

Scenario

An online store may retrieve product information and then pass that information to another function responsible for displaying it.

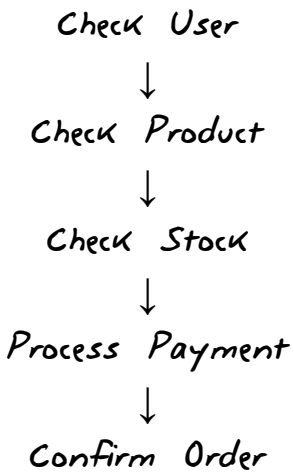
Conceptually:



3.2.9 Limitation of Multiple Callbacks

Callbacks are useful, but a program may become difficult to read when several asynchronous operations depend on one another.

Consider a shopping process:



Using many callbacks inside other callbacks can produce deeply nested code.

Example

```
checkUser(function () {

    checkStock(function () {

        processPayment(function () {

            confirmOrder(function () {
                console.log("Order Completed");
            });

        });

    });

});
```

Such deeply nested callback code is commonly called **callback hell.**

3.2.10 Callback Hell

Definition: Callback hell refers to a situation in which several callback functions are nested inside one another, producing code that is difficult to read, understand, and maintain. It commonly occurs when a sequence of asynchronous operations depends on the completion of earlier operations. Deep nesting can also make error handling more complicated. Promises provide a more structured method for representing and managing many asynchronous operations.

Scenario

An online shopping website may need to perform:

Login Verification
↓
Stock Verification
↓
Payment Processing
↓
Order Confirmation

If every operation is placed inside the callback of the previous operation, the program may become unnecessarily difficult to manage.

This problem encouraged wider use of **Promises.**

3.2.11 Promise

Definition: A Promise in JavaScript is an object that represents the eventual completion or failure of an asynchronous operation and the value that may result from that operation. A promise allows the program to start an operation and define what should happen when the operation

succeeds or fails. It provides a more structured alternative to deeply nested callbacks. Promises are widely used in modern JavaScript for handling server requests and other asynchronous operations.

Syntax

```
const promiseName = new Promise(function (resolve, reject) {

    // Asynchronous operation

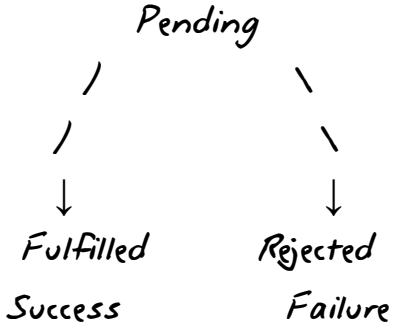
});
```

The Promise constructor commonly receives two functions:
resolve → Operation completed successfully
reject → Operation failed

3.2.12 States of a Promise

- A Promise can be in one of three states.
- **Pending** means the asynchronous operation has not yet completed.
- **Fulfilled** means the operation has completed successfully and a result is available.
- **Rejected** means the operation has failed and an error or reason for failure is available.

The state can be represented as:



Once a promise becomes fulfilled or rejected, its settled state does not change to the other state.

3.2.13 resolve()

Definition: resolve() is the function used inside a Promise to indicate that the asynchronous operation has completed successfully. A successful value may be supplied to resolve(). That value can later be processed by a .then() handler attached to the promise. Calling resolve() therefore represents successful completion of the promise operation.

Example

```
const resultPromise =
      new Promise(function (resolve, reject) {

            let resultAvailable = true;

            if (resultAvailable) {
                  resolve("Result Found");
            }
      });
```

Scenario

A college result portal successfully finds the student's result. The program can resolve the promise with the required result information.

3.2.14 reject()

Definition: reject() is the function used inside a Promise to indicate that the asynchronous operation has failed. It normally receives an error message or another value describing the reason for failure. A rejected promise can be handled using .catch(). This allows success and failure logic to be managed separately.

Example

```
const resultPromise =
      new Promise(function (resolve, reject) {
```

```
    let resultAvailable = false;

    if (resultAvailable) {
        resolve("Result Found");
    } else {
        reject("Result Not Available");
    }

});
```

Scenario

If a college server cannot find a result for the entered registration number, the promise may be rejected with an appropriate message.

3.2.15 The .then() Method

Definition: The .then() method is used to specify what should happen when a Promise is fulfilled successfully. It receives the successful value produced by the promise and passes it to a function for further processing. Multiple promise operations can also be connected using .then() calls. This produces more readable asynchronous code than deeply nested callbacks.

Example

```
resultPromise.then(function (message) {
    console.log(message);
});
```

If the promise is fulfilled:

Result Found

is displayed.

Scenario

After a shopping website successfully obtains product information, .then() can be used to display the received product details.

3.2.16 The .catch() Method

Definition: The .catch() method is used to handle a Promise that has been rejected or an error that occurs in a promise chain. It receives information describing the failure and allows the program to respond appropriately. This keeps failure-handling logic separate from normal successful processing. It is especially important when dealing with operations such as server communication.

Example

```
resultPromise.catch(function (error) {
        console.log(error);
});
```

If the promise is rejected:
Result Not Available
is displayed.

Scenario

If an online store fails to obtain product information, .catch() can display an error such as:
Unable to load product details.

3.2.17 The .finally() Method

Definition: The .finally() method specifies code that should execute after a Promise has settled, regardless of whether it was fulfilled or rejected. It is useful for operations that must occur after both successful and failed outcomes. For example, a loading indicator can be removed after a server operation has finished. The method does not normally process the successful value or failure reason itself.

Example

```
resultPromise.finally(function () {
        console.log("Operation Completed");
});
```

Scenario

A website display:

Loading...

while waiting for information.

Whether the request succeeds or fails, .finally () can be used to hide the loading message after completion.

3.2.18 Program Using a Promise

```
<!DOCTYPE html>
<html>

<head>
        <title>Student Result</title>
</head>

<body>

<h2>Student Result Portal</h2>

<button onclick="checkResult()">
        Check Result
</button>
```

```
<p id="message"></p>

<script>

function checkResult() {

    document.getElementById("message").textContent =
        "Searching for result...";

    const resultPromise =
        new Promise(function (resolve, reject) {

            setTimeout(function () {

                let resultAvailable = true;

                if (resultAvailable) {
                    resolve("Result: PASS");
                } else {
                    reject("Result not available");
                }

            }, 2000);

        });
    resultPromise
        .then(function (result) {

            document.getElementById("message").textContent =
                result;

        })
        .catch(function (error) {
```

```
                document.getElementById("message").textContent =
                    error;

            });

}

</script>

</body>

</html>
```

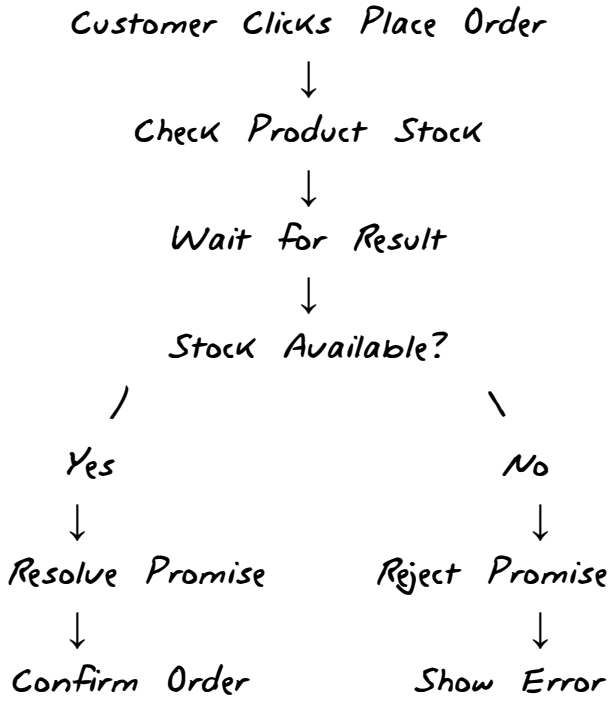
3.2.19 Callback and Promise

Callback	Promise
A function passed to another function	An object representing a future result
Common in asynchronous operations	Common in modern asynchronous JavaScript
Multiple nesting may become difficult to read	Supports structured chaining
Success and failure handling may be mixed	.then() and .catch() separate common success and failure handling
Suitable for simple asynchronous tasks	Suitable for larger asynchronous workflows

3.2.2.1 Website Scenario

Consider an online shopping website.

When the customer clicks **Place Order**, the following operations may occur:



JavaScript should not unnecessarily freeze the webpage while waiting for the stock verification.

A Promise can represent the future result of this operation.

If stock is available:

```
resolve("Order Confirmed");
```

If stock is unavailable:

```
reject("Product Out of Stock");
```

This is the fundamental reason asynchronous programming and promises are important in modern web applications.

3.2.22 Scenario-Based Questions

Question 1: A shopping website is waiting for product information from a server but should allow the user to continue interacting with the webpage. Which programming approach is required?
Answer: Asynchronous programming.

Question 2: A function must execute after another operation finishes. Which JavaScript concept can be used?
Answer: A callback function.

Question 3: Several callbacks are deeply nested inside one another and the program becomes difficult to read. What is this situation commonly called?
Answer: Callback hell.

Question 4: A website starts an asynchronous operation and needs to represent its future success or failure. Which JavaScript object is suitable?
Answer: A Promise.

Question 5: A student's result is successfully obtained. Which Promise function can indicate success?
Answer: resolve().

Question 6: The server fails to obtain the student's result. Which Promise function can indicate failure?
Answer: reject().

Question 7: Which method normally processes the successful value of a Promise?
Answer: .then().

Question 8: Which method is commonly used to handle a rejected Promise?

Answer: .catch().

3.2.23 Important Questions

1. What is asynchronous programming?

Asynchronous programming is an approach in which an operation can begin without requiring the remaining program to remain blocked until that operation finishes. The result can be processed later when it becomes available.

2. What is the difference between synchronous and asynchronous programming?

Synchronous programming normally executes operations sequentially and waits for one operation before proceeding to the next. Asynchronous programming allows other work to continue while waiting for a delayed operation.

3. What is a callback function?

A callback is a function passed to another function so that it can be executed at an appropriate time, often after another operation has completed.

4. What is callback hell?

Callback hell is a situation in which several callbacks are deeply nested, making the program difficult to read, maintain, and handle correctly.

5. What is a Promise?

A Promise is an object representing the eventual success or failure of an asynchronous operation and the value that may result from it.

6. What are the three states of a Promise?

The three states are **pending**, **fulfilled**, and **rejected**.

7. What is the purpose of resolve()?

resolve() indicates successful completion of a Promise and may provide the successful result.

8. What is the purpose of reject()?

reject() indicates failure of a Promise and may provide the reason for the failure.

9. What is the purpose of .then()?

.then() specifies processing that should occur after a Promise is successfully fulfilled.

10. What is the purpose of .catch()?

.catch() handles rejection or errors associated with a Promise.

3.2.24 Summary

Synchronous JavaScript normally performs operations sequentially, while asynchronous programming allows delayed operations to complete without unnecessarily blocking other program activities.

A **callback** is a function that can be executed after another operation completes. Callbacks are useful for simple asynchronous tasks, but deeply nested callbacks may create callback hell.

A **Promise** provides a structured representation of a future asynchronous result. A Promise begins in the pending state and later becomes either fulfilled or rejected. resolve() indicates success, reject() indicates failure, .then() processes successful results, and .catch() handles failures.

3.3 Async/Await, Fetch API and Form Validation

3.3.1 Introduction

In the previous topic, we learned that asynchronous operations may take time to complete and that JavaScript can represent such operations using **Promises**. Although .then() and .catch() are useful for handling promises, programs containing several asynchronous operations can still become difficult for beginners to read.

JavaScript provides **async and await** to write promise-based asynchronous code in a form that appears more sequential and easier to understand. The **Fetch API** allows JavaScript to request data from a server or web service. **Form validation** checks whether information entered by a user satisfies the required conditions before it is processed or submitted.

These concepts are widely used together in modern websites. For example, a student may enter a registration number, JavaScript may validate it, and then the website may asynchronously request the corresponding information from a server.

3.3.2 Need for Async/Await

Promises can be handled using .then() and .catch().

Example:

```
fetch("data.json")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.log(error));
```

This code is valid, but when several dependent operations are involved, multiple .then() statements may make the program harder for beginners to follow.

async and await provide a more readable form:

```
async function loadData() {
        const response = await fetch("data.json");
        const data = await response.json();

        console.log(data);
}
```

The code appears to execute one step after another even though asynchronous operations are being performed.

Scenario

A college portal must:

1. Request student details.
2. Wait for the response.
3. Convert the response into usable data.
4. Display the student's information.

Using async and await makes these steps easier to read and maintain.

3.3.3 The async Keyword

Definition: The **async keyword is used to declare a function as asynchronous and causes that function to return a Promise.** An async function can contain one or more await expressions that allow promise-based operations to be written in a more readable sequence. If the function successfully returns a value, JavaScript fulfills the returned Promise with that value. If an error is thrown inside the function, the returned Promise becomes rejected.

Syntax

```
async function functionName() {
        // Asynchronous statements
}
```

Example

```
async function showMessage() {
        return "Welcome Student";
}
```

Calling the function:

```
showMessage().then(function (message) {
        console.log(message);
});
```

Output:

Welcome Student

Even though "Welcome Student" looks like an ordinary return value, an async function returns it through a Promise.

Scenario

A university portal may define an asynchronous function called loadResult() because retrieving examination results from a server may require time.

```
async function loadResult() {
        // Server communication
}
```

3.3.4 The await Keyword

Definition: The **await keyword is used inside an async function to wait for the settlement of a Promise before continuing with the next statement in that async function.** If the Promise is fulfilled, await produces its fulfilled value. If the Promise is rejected, the rejection

behaves like an error and can be handled using try...catch. While the async function is waiting, JavaScript does not require the entire webpage to remain blocked.

Syntax

```
const result = await promise;
```

Example

```
function getMessage() {

        return new Promise(function (resolve) {

                setTimeout(function () {
                        resolve("Data Received");
                }, 2000);

        });
}
Using await:
async function displayMessage() {

        const message = await getMessage();

        console.log(message);
}

displayMessage();
```

After approximately two seconds:
Data Received
is displayed.

Scenario

An online store requests the current price of a product. The webpage can use await to wait for the price information inside an async function while the browser remains available for other activities.

3.3.5 async and await with Promises

async and await do not replace Promises. They provide another way of working with Promises.

Consider:

```
const result =
        new Promise(function (resolve) {

                resolve("Result Available");

        });
```

Using .then():

```
result.then(function (message) {
        console.log(message);
});
```

Using await:

```
async function displayResult() {

        const message = await result;

        console.log(message);
}

displayResult();
```

Both approaches work with the same Promise.

Comparison

Promise Style	Async/Await Style
Uses .then()	Uses await
Uses .catch() for errors	Commonly uses try...catch
Useful for chaining	Often easier to read sequentially
Promise-based	Also Promise-based

3.3.6 Error Handling with try...catch

Definition: A try...catch statement is an error-handling structure in which JavaScript attempts to execute the statements inside the try block and transfers control to the catch block if an error is thrown. It is commonly used with async and await because an awaited Promise may be rejected. This allows successful processing and error handling to be kept clearly separated.

Syntax

```
try {
      // Statements that may produce an error
}
catch (error) {
      // Handle the error
}
```

Example

```
async function loadData() {

      try {

            const result = await somePromise;

            console.log(result);
```

```
        }
        catch (error) {

                console.log("Operation failed");

        }
}
```

Scenario

If an online shopping website cannot obtain product information because the server is unavailable, the catch block can display:

- Unable to load product details.
- instead of allowing the application to fail silently.

3.3.7 Fetch API

Definition: The **Fetch API is a browser interface used by JavaScript to make network requests and receive responses from resources such as web servers and web APIs.** The fetch() function starts a request and returns a Promise representing the response. Developers can then process the received information in formats such as JSON or text. Fetch is widely used in modern web applications for asynchronous communication.

Syntax

```
fetch(url);
```

Example:

```
fetch("https://jsonplaceholder.typicode.com/users/1");
```

Because fetch() returns a Promise, it can be used with:

.then()

or:

await

3.3.8 Why Is the Fetch API Required?

Consider a shopping website. Product information may be stored on a server rather than directly inside the HTML page.

When a customer chooses a product, JavaScript may need to request:

Product Name

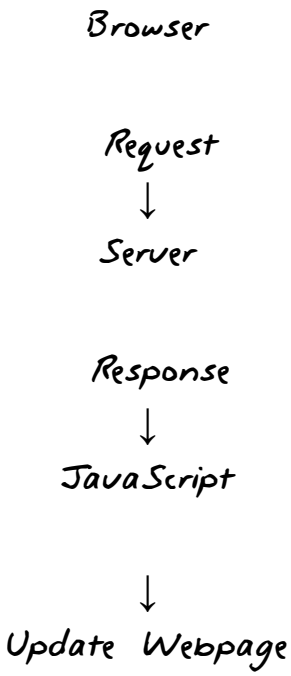
Price

Stock

Rating

The Fetch API allows the browser to request this information without manually reloading the complete webpage.

Conceptually:



Scenario

A student enters an ID on a college portal. JavaScript sends a request for the corresponding student data. When the information arrives, only the result section of the webpage is updated.

3.3.9 Fetch Example

```
fetch("https://jsonplaceholder.typicode.com/users/1")
        .then(function (response) {

                return response.json();

        })
        .then(function (data) {

                console.log(data.name);

        });
```

A possible output is:
Leanne Graham

The first .then() receives the server response.
response.json()

reads the JSON body and itself returns a Promise.

The second .then() receives the converted JavaScript data.

3.3.10 Fetch Using Async/Await

The same operation can be written more clearly using async and await.

```
async function loadUser() {

        const response =
                await fetch(
                        "https://jsonplaceholder.typicode.com/users/1"
```

```
        );

    const data =
        await response.json();

    console.log(data.name);
}

loadUser();
```

How It Works

The statement:

```
await fetch(...)
```

waits for the network response inside the async function.

The statement:

```
await response.json()
```

waits until the response body has been read and converted from JSON into JavaScript data.

The program can then access:
`data.name`

3.3.11 Checking the Fetch Response

A network request can receive an HTTP error response such as 404. For this reason, the response should commonly be checked using:

```
response.ok
```

Example

```
async function loadUser() {
    try {

        const response =
            await fetch(
                "https://jsonplaceholder.typicode.com/users/1"
            );

        if (!response.ok) {
            throw new Error("Data not found");
        }

        const data =
            await response.json();

        console.log(data.name);

    }
    catch (error) {

        console.log("Unable to load data");

    }
}
```

Scenario

A library website requests information about a book. If the server cannot find the requested book, the application can display an appropriate message instead of attempting to process missing data.

3.3.12 Form Validation

Definition: Form validation is the process of checking user-entered information against specified rules before the information is accepted or submitted for further processing. Validation helps ensure that required fields are completed and that values follow appropriate formats or ranges. Web forms can use built-in HTML validation features as well as JavaScript for additional validation logic. Client-side validation improves usability but should not be considered a replacement for server-side validation when data security and correctness are important.

Need for Form Validation

Suppose an admission form asks for:

Student Name

Email

Age

Mobile Number

Without validation, a user could submit:

Name: blank

Email: abc

Age: -10

Such information is incomplete or invalid.

Validation checks the data before proceeding.

3.3.13 HTML Form Validation

HTML provides several attributes for basic form validation.

Common examples include:

required

type="email"

min

max

minlength

maxlength

pattern

Example

```
<input
        type="email"
        id="email"
        required >
```

The browser checks whether the field is filled and whether the entered value follows the basic structure expected for an email address.

Scenario

A college admission form requires every applicant to enter an email address. The required attribute prevents an empty submission, while type="email" provides basic email-format validation.

3.3.14 JavaScript Form Validation

Definition: JavaScript form validation uses program logic to examine form values and determine whether they satisfy application-specific requirements before further processing occurs. It is useful when validation conditions are more specific than the built-in HTML rules. JavaScript can read form values, test conditions, display suitable error messages, and prevent further processing when the values are invalid.

Example

```
<input type="number" id="age">
<button onclick="validateAge()">
        Submit
</button>
```

```
<p id="message"></p>
JavaScript:
function validateAge() {

    const age =
        Number(
            document.getElementById("age").value
        );

    if (age < 18) {

        document.getElementById("message").textContent =
            "Age must be 18 or above.";

    }
    else {

        document.getElementById("message").textContent =
            "Valid age.";

    }
}
```

Scenario

An online registration portal accepts applications only from users aged 18 or above. JavaScript checks the age immediately and displays an appropriate message.

3.3.15 Why Both HTML and JavaScript Validation Are Useful

HTML validation is suitable for common rules such as:

Required field

Email format

Minimum value

Maximum value

JavaScript validation is useful for more specific application rules such as:

Password and confirm password must match

Age must satisfy an admission rule

Marks must fall within a specific range

Two fields must be compared

A practical website often uses both.

3.3.16 Website Program

The following program combines:

- Form validation
- async
- await
- Fetch API
- Error handling

It allows the user to enter a user ID and then requests user information.

```
<!DOCTYPE html>
<html>

<head>
      <title>User Information</title>
</head>

<body>

<h2>User Information Portal</h2>
```

```
<label for="userId">
    Enter User ID (1 to 10):
</label>

<input
    type="number"
    id="userId"
    min="1"
    max="10"
    required >

<button onclick="loadUser()">
    Search
</button>

<p id="result"></p>

<script>
async function loadUser() {

    const userId =
        Number(
            document.getElementById("userId").value
        );

    const result =
        document.getElementById("result");


    if (userId < 1userId > 10) {

        result.textContent =
            "Please enter a User ID from 1 to 10.";
```

```
        return; }
result.textContent =
        "Loading user information...";
try {

        const response =
                await fetch(
                        "https://jsonplaceholder.typicode.com/users/"
                        + userId
                );
        if (!response.ok) {

                throw new Error(
                        "User information not found"
                );
        }
        const data =
                await response.json();
        result.innerHTML =
                "Name: " + data.name +
                "<br>Email: " + data.email +
                "<br>City: " +
                data.address.city;

}
catch (error) {

        result.textContent =
                "Unable to load user information.";

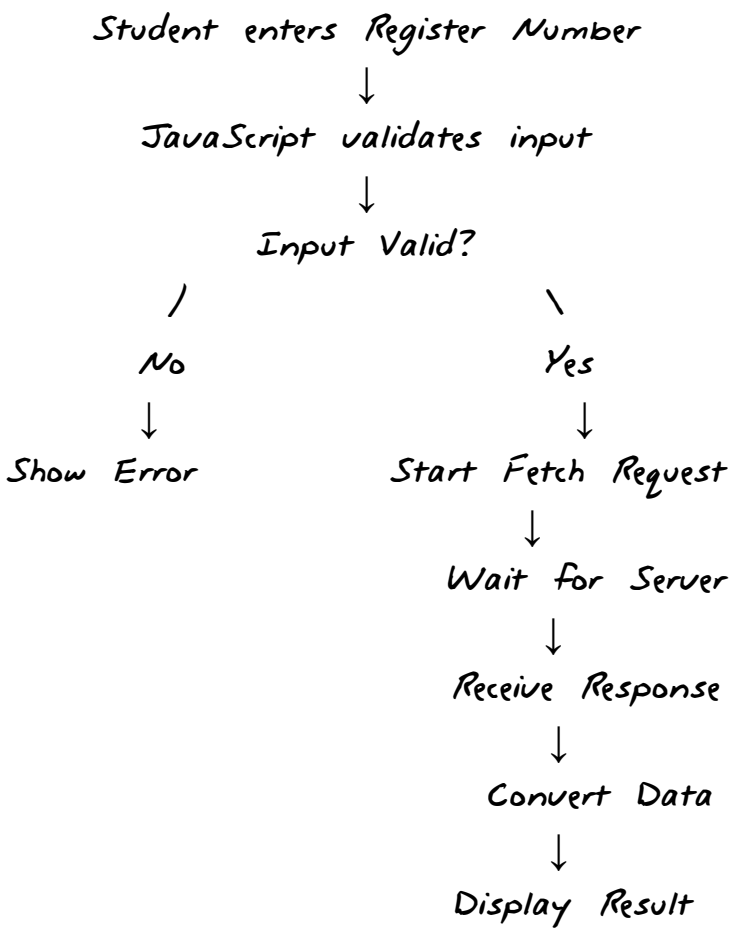
}
```

```
}
</script>
</body>
</html>
```

3.3.17 Scenario

Consider a college student-information portal.

The working principle may be represented as:



The webpage remains responsive while the request is being processed. This demonstrates the practical relationship between **form validation, async/await, and the Fetch API.**

3.3.18 Comparison

Concept	Main Purpose	Website Example
async	Declares an asynchronous function	Function that loads student data
await	Waits for a Promise inside an async function	Wait for server response
try...catch	Handles errors	Handle failed request
Fetch API	Requests resources over the network	Request product information
response.json()	Reads JSON response data	Convert server response
Form Validation	Checks user input	Check age, email or registration number

3.3.19 Scenario-Based Questions

Question 1: A university website retrieves a student's result from a server, and the operation may take a few seconds. Which programming technique should be used?
Answer: Asynchronous programming using mechanisms such as async/await.

Question 2: Which keyword declares a function that can conveniently use await?
Answer: async.

Question 3: Which keyword is used to wait for a Promise inside an async function?
Answer: await.

Question 4: An online shopping website needs to request product information from a web service. Which browser API can be used?
Answer: Fetch API.

Question 5: A Fetch request returns JSON data. Which method is commonly used to read it?
Answer: response.json().

Question 6: A registration page must ensure that the age entered by a user is at least 18. Which concept is required?
Answer: Form validation.

Question 7: A network request fails while an async function is running. Which structure can be used to handle the error?
Answer: try...catch.

3.3.20 Important Questions

1. What is an async function?
An async function is a function declared using the async keyword. It always returns a Promise and can use await to work conveniently with asynchronous operations.

2. What is the purpose of await?
await waits for the settlement of a Promise inside an async function and produces its fulfilled value when successful.

3. Do async/await replace Promises?
No. Async/await provide a more readable syntax for working with Promises.

4. What is the Fetch API?

The Fetch API is a browser interface that allows JavaScript to make network requests and receive responses asynchronously.

5. What does fetch() return?

fetch() returns a Promise representing the response to the request.

6. What does response.json() do?

It reads the response body as JSON and returns a Promise that fulfills with the corresponding JavaScript data.

7. What is form validation?

Form validation checks whether information entered by a user satisfies the specified rules before further processing or submission.

8. Why is JavaScript validation useful?

JavaScript validation can implement application-specific conditions and provide immediate feedback to users.

9. Is client-side validation sufficient for secure applications?

No. Client-side validation improves user experience, but important data should also be validated on the server.

3.3.21 Summary

async and await provide a readable method for working with Promise-based asynchronous operations. An async function returns a Promise, while await allows the function to wait for the result of another Promise without unnecessarily blocking the entire webpage. Errors are commonly handled using try...catch.

The **Fetch API** allows JavaScript to request information from servers and web APIs. The returned response can be processed in formats such as JSON.

Form validation checks user input before further processing. HTML provides built-in validation features, while JavaScript supports more application-specific validation rules.
Together, these concepts are commonly used in modern websites to validate user input, communicate with servers asynchronously, process received information, and update webpage content dynamically.

3.4 XML Syntax and Namespaces

3.4.1 Introduction

Web applications frequently exchange structured information such as student details, product information, employee records, transaction data, and configuration settings. **XML (Extensible Markup Language)** provides a text-based method for representing such information using meaningful tags.

Unlike HTML, XML does not contain a fixed collection of predefined tags. Developers create tags according to the type of information being represented. Therefore, XML follows strict syntax rules so that different applications can interpret the document correctly.

When XML documents combine information from different applications, the same element name may be used with different meanings. **XML namespaces** solve this problem by uniquely identifying groups of XML elements.

3.4.2 XML

Definition: XML, or Extensible Markup Language, is a text-based markup language designed to represent, store, and exchange structured information. XML allows users to define meaningful tags according to the nature of the data being represented. It organizes information hierarchically using elements, attributes, and values. XML emphasizes the structure and meaning of data rather than its visual presentation.

Example

```
<student>
    <name>Ananya</name>
    <department>CSE</department>
    <marks>85</marks>
```

</student>

Here:

student, name, department, and marks are XML elements.

The values are:

Ananya, CSE, and 85.

Scenario

A university may exchange student details between an admission system and an examination system using XML.

<student>

<rollno>101</rollno>

<name>Rahul</name>

<department>CSE</department>

</student>

Because each tag describes the information it contains, another system can understand the data easily.

3.4.3 Need for XML Syntax Rules

XML documents may be exchanged between different applications and computer systems. Therefore, the document must follow a predictable structure.

For example, the following is incorrect:

<student>

<name>Ananya

<marks>85</student>

The tags are not properly closed or nested.

A computer program may not be able to process such a document correctly.

XML therefore defines syntax rules concerning:

- XML declaration
- Root element
- Opening and closing tags
- Proper nesting
- Case sensitivity
- Attribute values
- Special characters

A document that satisfies the required XML syntax rules is described as **well-formed XML**.

3.4.4 Well-Formed XML Document

Definition: A **well-formed XML document is an XML document that satisfies the fundamental syntactic rules defined by the XML specification.** It must contain properly formed elements, correct nesting, quoted attribute values, and a single document element or root element. Element names and corresponding closing tags must also match exactly. A document that violates these rules cannot be treated as a correctly formed XML document.

Correct Example

```
<student>
    <name>Ananya</name>
    <marks>85</marks>
</student>
```

Incorrect Example

```
<student>
    <name>Ananya</Name>
</student>
```

The opening tag is:

<name>

but the closing tag is:

</Name>

Since XML is case-sensitive, these are considered different names.

3.4.5 XML Declaration

Definition: The **XML declaration is an optional statement placed at the beginning of an XML document that provides information about the XML version and, when specified, the character encoding used by the document.** It normally begins with <?xml and ends with ?>. When present, it must appear at the beginning of the document. The declaration helps XML processors interpret the document correctly.

Syntax

```
<?xml version="1.0" encoding="UTF-8"?>
```

Here:

version="1.0" specifies the XML version.

encoding="UTF-8" specifies the character encoding.

Example

```
<?xml version="1.0" encoding="UTF-8"?>
<student>
    <name>Ananya</name>
</student>
```

Scenario

A university exchanges student names in multiple languages. Using UTF-8 helps the XML document represent a wide range of characters consistently.

3.4.6 XML Elements

Definition: An XML element is a structural component of an XML document that is normally represented by a start tag, content, and an end tag. Elements are used to identify and organize individual pieces of information. An element may contain text, attributes, or other child elements. Developers choose element names according to the meaning of the data.

Syntax

```
<elementName>value</elementName>
```

Example

```
<name>Ananya</name>
```

Here:

<name> is the opening tag.

Ananya is the element content.

</name> is the closing tag.

Scenario

A library system can represent the title of a book as:

<title>Web Programming</title>

The element name title clearly describes the stored value.

3.4.7 Root Element

Definition: The **root element is the single outermost element that contains all other elements of an XML document.** Every well-formed XML document must have exactly one root element. Child elements are placed inside this root to form a hierarchical structure. The root element therefore represents the complete document entity.

Correct Example

```
<student>
    <name>Rahul</name>
    <department>CSE</department>
</student>
```

Here:

<student>

is the root element.

Incorrect Example

<name>Rahul</name>

<department>CSE</department>

There are two top-level elements and no single root element.

Corrected Version

<student>

<name>Rahul</name>

<department>CSE</department>

</student>

Scenario

A product XML file may use:

<products>

as the root element and place several <product> elements inside it.

3.4.8 Opening and Closing Tags

Definition: An **XML element normally begins with a start tag and ends with a corresponding end tag having exactly the same element name.** The end tag contains a forward slash before the element name. Correct tag matching is necessary for an XML document to be well formed. XML does not automatically infer omitted closing tags in the way some HTML processing may.

Correct

<department>CSE</department>

Incorrect

<department>CSE

Empty Element

An element containing no content can be written as:

<photo></photo>

or in empty-element form:

<photo/>

Both represent an empty element.

Scenario

If a student has not uploaded a photograph, an XML record might contain:

<photo/>

3.4.9 Proper Nesting of Elements

Definition: XML elements must be properly nested so that an element opened inside another element is closed before its parent element is closed. Elements cannot overlap one another. Proper nesting creates a clear parent-child hierarchy that XML processors can interpret correctly. Incorrectly nested elements make the document not well formed.

Correct

<student>

```
    <name>Ananya</name>
</student>
```

Incorrect

```
<student>
    <name>Ananya
</student>
    </name>
```

Scenario

In a college XML document, the student's details must remain inside the corresponding <student> element so that information belonging to different students is not mixed.

3.4.10 XML is Case-Sensitive

Definition: XML is case-sensitive, which means uppercase and lowercase letters are treated as different characters in element and attribute names. Therefore, an opening tag and its closing tag must use exactly the same capitalization. For example, <name> and <Name> represent different element names. Consistent capitalization is essential when designing XML documents.

Correct

```
<student>
    <name>Rahul</name>
</student>
```

Incorrect

```
<student>
    <name>Rahul</Name>
</student>
```

Scenario

If a student-information application expects:

<studentId>

then:

<StudentId>

may be interpreted as a different element.

3.4.11 XML Attributes

Definition: An **XML attribute provides additional information about an element and is written inside the element's start tag as a name-value pair.** Attribute values must be enclosed within quotation marks. Attributes are generally suitable for metadata or supplementary information, while elements are commonly used for the main structured content. Attribute names must also obey XML naming and namespace rules.

Syntax

```
<element attribute="value">
```

Example

```
<student id="101">
    <name>Ananya</name>
</student>
```

Here:

id="101"

is an attribute.

Scenario

A university may identify each student record using:

<student id="101">

while storing the student's descriptive information as child elements.

3.4.12 XML Comments

Definition: An **XML comment is descriptive text included in an XML document for human readers and is not treated as normal document content.** Comments begin with <!-- and end with -->. They may be used to explain sections of a document or provide development information. The character sequence -- cannot appear inside an XML comment.

Example

```
<!-- Student admission information -->
<student>
```

```
    <name>Ananya</name>
</student>
```

Scenario

A developer maintaining a college XML configuration file may add comments to identify sections containing admission, examination, or placement information.

3.4.13 Special Characters in XML

Some characters have special meaning in XML markup and therefore cannot always be written directly as ordinary text.

Common predefined XML entities include:

Character	Entity Reference
<	<
>	>
&	&
"	"
'	'

Example

Suppose a company name is:

ABC & XYZ

Writing:

```
<company>ABC & XYZ</company>
```

is not correct because & begins an entity reference.

It should be:

<company>ABC & XYZ</company>

Scenario

An e-commerce product description containing:

Books & Stationery

can be represented as:

<category>Books & Stationery</category>

3.4.14 Complete XML Example

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student id="101">
            <name>Ananya</name>
            <department>CSE</department>
            <marks>85</marks>
      </student>
  <student id="102">
            <name>Rahul</name>
            <department>ECE</department>
            <marks>78</marks>
      </student>
</students>
```

3.4.15 Need for XML Namespaces

XML allows developers to create their own element names. This flexibility can create a problem when XML information from different sources is combined.

Consider two applications.

A college system may use:

<table>

to represent a database table.

A furniture catalogue may also use:

<table>

to represent a physical table.

The same element name has two different meanings.

XML namespaces provide a mechanism for distinguishing such names.

3.4.16 XML Namespace

Definition: An XML namespace is a mechanism for uniquely qualifying element and attribute names in an XML document so that naming conflicts can be avoided when vocabularies from different sources are combined. A namespace is identified using a namespace name, normally written as a URI. Elements can be associated with that namespace through a prefix or through a default namespace declaration. Namespaces therefore allow identical local element names to be used with different meanings without ambiguity.

Syntax

xmlns:prefix="namespace-URI"

Example:

xmlns:stu="https://example.com/student"

Here:

xmlns declares a namespace.

stu is the namespace prefix.

https://example.com/student is the namespace name.

3.4.17 Namespace Prefix

Definition: A **namespace prefix is a short name associated with an XML namespace and written before an element or attribute name followed by a colon.** The prefix makes it possible to determine which namespace a particular qualified name belongs to. Different prefixes may technically refer to the same namespace name, so the namespace URI—not the chosen prefix—is what identifies the namespace. Prefixes are mainly used to make names easier to read and distinguish.

Syntax

prefix:elementName

Example

<stu:name>Ananya</stu:name>

Here:

stu is the namespace prefix.

name is the local element name.

Together:

stu:name

forms a qualified name.

3.4.18 Namespace Declaration

A namespace prefix must first be associated with a namespace name.

Example:

```
<stu:student
        xmlns:stu="https://example.com/student">
```

Now elements beginning with:

stu:

belong to the declared student namespace.

Example

```
<stu:student
        xmlns:stu="https://example.com/student">

        <stu:name>Ananya</stu:name>
        <stu:department>CSE</stu:department>

</stu:student>
```

Important Point

The namespace URI:

https://example.com/student

acts as an identifier.

It does **not necessarily mean that the browser must visit that web address and download a webpage.**

3.4.19 Why Namespaces Are Required – Beginner Scenario

Suppose a university portal combines student information and library information.

Both systems use an element called:

<id>

The student system means:

Student ID

while the library system means:

Book ID

Without namespaces:

<id>101</id>

<id>B501</id>

it may be unclear which ID belongs to which system.

With namespaces:

<stu:id>101</stu:id>

<lib:id>B501</lib:id>

the meaning becomes clear.

3.4.20 Namespace Conflict Example

```
<?xml version="1.0" encoding="UTF-8"?>
<records
        xmlns:stu="https://example.com/student"
        xmlns:lib="https://example.com/library">
        <stu:id>101</stu:id>
        <stu:name>Ananya</stu:name>
        <lib:id>B501</lib:id>
        <lib:title>Web Programming</lib:title>
</records>
```

Here:

stu:id

represents a student ID.

lib:id

represents a library book ID.

Although both use the local name id, their namespaces distinguish their meanings.

3.4.21 Default Namespace

Definition: A default namespace is an XML namespace declared without a prefix and automatically applied to unprefixed element names within its scope. It is declared using the xmlns attribute rather than xmlns:prefix. A default namespace reduces the need to repeatedly write

a prefix when most elements belong to the same namespace. By default, it applies to element names, not automatically to unprefixed attribute names.

Syntax

xmlns="namespace-URI"

Example

<student

xmlns="https://example.com/student">

<name>Ananya</name>

<department>CSE</department>

</student>

Here, student, name, and department belong to the declared default namespace.

Scenario

If an entire student-information XML document belongs to one university data vocabulary, a default namespace can avoid repeatedly writing:

stu:

before every element.

3.4.22 Prefix Namespace and Default Namespace

Prefix Namespace

```
<stu:student
        xmlns:stu="https://example.com/student">
        <stu:name>Ananya</stu:name>
</stu:student>
```

Default Namespace

```
<student
        xmlns="https://example.com/student">
    <name>Ananya</name>
</student>
```

Both associate elements with an XML namespace.

The first explicitly displays the prefix.

The second uses the namespace as the default for unprefixed elements in its scope.

3.4.23 Complete XML Document Using Namespaces

```
<?xml version="1.0" encoding="UTF-8"?>
<college
        xmlns:stu="https://example.com/student"
        xmlns:lib="https://example.com/library">
b<stu:student>
```

```
        <stu:id>101</stu:id>
        <stu:name>Ananya</stu:name>
        <stu:department>CSE</stu:department>
    </stu:student>
     <lib:book>
        <lib:id>B501</lib:id>
        <lib:title>Web Programming</lib:title>
        <lib:status>Available</lib:status>
    </lib:book>
</college>
```

3.4.24 XML Syntax and Namespace

Concept	Purpose	Example
XML declaration	Provides XML version and encoding	<?xml version="1.0"?>
Element	Represents information	<name>Ananya</name>
Root element	Contains the complete XML document	<students>
Attribute	Provides additional element information	id="101"
Well-formed XML	Follows XML syntax rules	Correct nesting and matching tags
Namespace	Distinguishes XML names from different vocabularies	Student vs Library
Namespace prefix	Short label associated with a namespace	stu:
xmlns	Declares a namespace	xmlns:stu="..."

Default namespace	Applies namespace without a prefix	xmlns="..."

3.4.25 Scenario-Based Questions

Question 1: A university XML document contains student name, department, and marks. Which language can represent this information using meaningful tags?

Answer: XML.

Question 2: An XML document contains several student elements. Which element should contain all of them?

Answer: A single root element such as <students>.

Question 3: An XML document uses <name> as the opening tag and </Name> as the closing tag. Is it correct?

Answer: No. XML is case-sensitive, so the opening and closing names must match exactly.

Question 4: A library wants to attach a unique code to a book element. Which XML feature can be used?

Answer: An XML attribute, for example:

<book id="B101">

Question 5: A university system and a library system both use the element <id> with different meanings. Which XML concept solves this naming conflict?

Answer: XML namespaces.

Question 6: Which attribute is used to declare an XML namespace?

Answer: xmlns.

Question 7: What does stu represent in <stu:name>?

Answer: It is the namespace prefix.

Question 8: A document mainly uses elements from one namespace and the developer does not want to repeat a prefix. Which type of namespace is suitable?

Answer: A default namespace.

3.4.26 Important Questions

1. What is XML?

XML is a text-based markup language used to represent, store, and exchange structured information. It allows developers to define meaningful tags according to the data being represented.

2. What is a well-formed XML document?

A well-formed XML document satisfies XML syntax rules, including a single root element, correctly nested elements, matching start and end tags, and properly quoted attribute values.

3. What is an XML element?

An XML element is a structural component normally consisting of a start tag, content, and an end tag. It represents a particular item of information.

4. What is a root element?

A root element is the single outermost element that contains all other elements in an XML document.

5. Is XML case-sensitive?

Yes. XML treats uppercase and lowercase letters as different characters, so <name> and <Name> are different element names.

6. What is an XML attribute?

An XML attribute is a name-value pair written in an element's start tag to provide additional information about that element.

7. What is an XML namespace?

An XML namespace is a mechanism for uniquely qualifying element and attribute names so that naming conflicts can be avoided when XML vocabularies are combined.

8. Why are XML namespaces required?

Namespaces distinguish elements that have the same local name but belong to different applications or vocabularies.

9. What is a namespace prefix?

A namespace prefix is a short identifier associated with a namespace and written before an element name followed by a colon.

10. What is a default namespace?

A default namespace is a namespace declared without a prefix and applied to unprefixed element names within its scope.

3.4.27 Summary

XML is used to represent and exchange structured data using developer-defined elements. XML documents follow strict syntax rules: they must have a single root element, correctly matched and nested tags, quoted attribute values, and consistent case.

A correctly structured XML document is described as **well formed**. XML elements represent information, attributes provide additional information, and entity references are used for certain special characters.

When XML data from different applications is combined, identical element names may create naming conflicts. **XML namespaces** solve this problem by associating element names with uniquely identified namespaces. Namespaces may use prefixes such as stu: and lib:, or a default namespace may be declared when most elements belong to the same vocabulary.

3.5 XML Schema and XML Parsers

3.5.1 Introduction

In the previous topic, we learned how XML represents structured information using user-defined elements and attributes. However, XML syntax alone does not specify which elements must appear, what type of data they should contain, or how many times an element may occur.

For example, a college may require every student record to contain a roll number, name, department, and marks. It may also require marks to be numeric rather than ordinary text. **XML Schema** is used to define such structural and data rules.

After an XML document is created, a software component must read and interpret it. This component is known as an **XML parser**. XML parsers convert XML text into a form that a program can access and process.

3.5.2 Need for XML Schema

Consider the following XML:

```
<student>
    <name>Ananya</name>
    <marks>Excellent</marks>
</student>
```

This XML may be **well formed** because its tags are properly written and nested. However, if a college system expects marks to be a number, the value Excellent is not acceptable.

Similarly, another document may contain:

<student>

<name>Ananya</name>

</student>

The XML is still well formed, but the required marks element is missing.

Therefore, a mechanism is required to specify:

- which elements are allowed;
- which elements are compulsory;
- the order of elements;
- the data type of values;
- which attributes are allowed;
- how many times an element may occur.

XML Schema provides these rules.

3.5.3 XML Schema

Definition: An **XML Schema is a formal description of the structure, content, and data constraints of a class of XML documents.** It specifies which elements and attributes may occur, their relationships, their data types, and restrictions on their values. XML Schema documents are themselves written using XML syntax. The W3C XML Schema language is commonly referred to as **XSD, or XML Schema Definition.**

Example

XML contains the actual data:

<marks>85</marks>

XML Schema defines the rule:

marks must contain an integer value.

Therefore:

XML → Contains data

XSD → Defines rules for that data

Scenario

A university receives student records from several colleges. To ensure that every record follows the same structure, the university can provide one XML Schema defining required fields such as student ID, name, department, and marks.

3.5.4 XML Schema Document

An XML Schema document commonly uses the extension:

.xsd

For example:

student.xsd

A simple schema begins with the <xs:schema> element.

```
<?xml version="1.0" encoding="UTF-8"?>

<xs:schema

      xmlns:xs="http://www.w3.org/2001/XMLSchema">

</xs:schema>
```

The prefix:

xs:

identifies elements belonging to the XML Schema vocabulary.

3.5.5 The <xs:schema> Element

Definition: The **<xs:schema> element is the root element of an XML Schema document and contains the declarations and definitions that describe valid XML documents.** It establishes the XML Schema vocabulary through the namespace http://www.w3.org/2001/XMLSchema. Elements, attributes, data types, and structural rules are normally defined inside this element. It therefore forms the outer container of an XSD document.

Example

```
<xs:schema
        xmlns:xs="http://www.w3.org/2001/XMLSchema">
</xs:schema>
```

Scenario

A college can create student.xsd and place all rules related to student XML records inside the <xs:schema> element.

3.5.6 Declaring an Element

Definition: An **element declaration in XML Schema specifies an XML element that may appear in documents governed by the schema.** The declaration can define the element's name, data type, occurrence restrictions, and other structural requirements. Simple element declarations are commonly written using <xs:element>. These declarations allow an XML processor to determine whether an element is being used correctly.

Syntax

```
<xs:element name="elementName" type="dataType"/>
```

Example

```
<xs:element
        name="name"
        type="xs:string"/>
```

This rule states that:

Element name → name

Data type → string

Scenario

A student XML document may require the student's name to contain text. The schema can therefore define the name element as xs:string.

3.5.7 XML Schema Data Types

Definition: An XML Schema data type defines the kind of value that an XML element or attribute is permitted to contain. XSD provides predefined data types for common values such as text, integers, decimal numbers, Boolean values, and dates. Data types help detect values that do not satisfy the expected format. They make XML documents more precise than documents in which every value is treated simply as text.

Data types include:

XSD Data Type	Used For	Example
xs:string	Text	Ananya
xs:integer	Whole numbers	85
xs:decimal	Decimal numbers	87.5
xs:boolean	True/false values	true
xs:date	Dates	2026-08-13

Example

```
<xs:element
        name="marks"
        type="xs:integer"/>
```

Scenario

A college result system expects marks to be a whole number. Defining the element as xs:integer prevents values such as:

<marks>Excellent</marks>

from satisfying that schema rule.

3.5.8 Simple Type

Definition: A simple type in XML Schema describes an element or attribute whose value contains text-based data but does not contain child elements or attributes of its own. Predefined XSD types such as string, integer, decimal, and date are examples of simple types. Developers can also define restrictions on simple values when required. Simple types are suitable for individual values such as names, marks, prices, and dates.

Example

```
<xs:element
        name="name"
        type="xs:string"/>
```

The name element contains one simple textual value.

Scenario

In a library XML document:

```
<title>Web Programming</title>
```

the book title can be represented using the simple type xs:string.

3.5.9 Complex Type

Definition: A complex type in XML Schema describes an element that can contain child elements, attributes, or a structured combination of both. It is used when an XML element represents a complete entity composed of several related values. Complex types make it possible to define hierarchical structures such as students, products, customers, and orders. They are commonly declared using the <xs:complexType> element.

Example

A student contains several child elements:

```
<student>
        <name>Ananya</name>
        <department>CSE</department>
        <marks>85</marks>
```

</student>

Therefore, student requires a complex structure.

XSD Example

```
<xs:element name="student">
    <xs:complexType>
        <xs:sequence>
            <xs:element
                name="name"
                type="xs:string"/>
            <xs:element
                name="department"
                type="xs:string"/>
            <xs:element
                name="marks"
                type="xs:integer"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

Scenario

A college portal receives complete student records containing several fields. A complex type can define how those related elements must appear together.

3.5.10 The <xs:sequence> Element

Definition: The <xs:sequence> element specifies that its child elements are expected to occur in the order defined by the schema. It is commonly used inside a complex type when an XML structure contains several related child elements. The sequence therefore controls both the permitted children and their ordering. This helps applications receive XML data in a predictable structure.

Example

```
<xs:sequence>
  <xs:element
            name="name"
            type="xs:string"/>
  <xs:element
            name="department"
            type="xs:string"/>
</xs:sequence>
```

The expected order is:

name

department

Scenario

A university may standardize student records so that the name appears before the department in every XML document.

3.5.11 XML Schema Attributes

Definition: An attribute declaration in XML Schema specifies an attribute that may appear on an XML element and defines its permitted type or usage. Attributes are usually declared using <xs:attribute>. A schema may indicate whether an attribute is optional or required. Attribute validation ensures that metadata associated with an element follows the expected rules.

Example XML

```
<student id="101">
```

XSD Declaration

```
<xs:attribute
        name="id"
        type="xs:integer"
        use="required"/>
```

The rule:

use="required"

means that the id attribute must be present.

Scenario

A college system may require every student record to contain a unique numeric student ID.

3.5.12 minOccurs and maxOccurs

Definition: The minOccurs and maxOccurs attributes in XML Schema specify the minimum and maximum number of times an element may appear. They are used to describe optional, compulsory, or repeated elements. The default value for each is normally 1 when not otherwise specified. The value unbounded for maxOccurs allows an element to repeat without a schema-defined upper limit.

Example

```
<xs:element
        name="student"
        minOccurs="1"
        maxOccurs="unbounded"/>
```

This means:

At least one student is required.

Any number of additional student elements are allowed.

Scenario

A college XML file may contain records for many students. maxOccurs="unbounded" allows the schema to accept multiple <student> elements.

3.5.13 Complete XML Schema Example

Consider the following XML document:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<student id="101">
    <name>Ananya</name>
    <department>CSE</department>
     <marks>85</marks>
</student>
```

A corresponding beginner-level XSD is:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
    xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="student">
 <xs:complexType>
                <xs:sequence>
       <xs:element
                                name="name"
                                type="xs:string"/>

                          <xs:element
                                name="department"
                                type="xs:string"/>
                          <xs:element
                                name="marks"
                                type="xs:integer"/>
```

```
</xs:sequence>
<xs:attribute
                    name="id"
                    type="xs:integer"
                    use="required"/>
</xs:complexType>
  </xs:element>
</xs:schema>
```

3.5.14 Valid XML Document

Definition: *A valid XML document is a well-formed XML document that also conforms to the structural and data constraints defined by an associated schema or other validation grammar.* Well-formedness checks the basic XML syntax, while validation checks additional document rules such as element names, order, data types, and required attributes. Therefore, every valid XML document must first be well formed, but a well-formed document is not necessarily valid against a particular schema.

Comparison

Well-Formed XML	Valid XML
Follows XML syntax rules	Follows XML syntax and schema rules
Tags correctly nested	Elements also satisfy XSD requirements
One root element	Correct data types and required fields

Does not require XSD validation	Validation requires a schema or grammar

Scenario

A student record may have perfectly matched XML tags but still fail validation if the schema requires a numeric student ID and the document contains letters.

3.5.15 XML Parser

Definition: An **XML parser is a software component that reads XML text, checks its syntactic structure, and makes its elements, attributes, and content available to a program for processing.** A parser identifies the hierarchical relationships among XML elements and normally reports an error when the document is not well formed. Some parsers can also perform validation against a schema when configured with validation support. XML parsing is therefore the process that converts XML markup into information that application code can use.

Scenario

Suppose a server returns:

```
<student>
    <name>Ananya</name>
    <marks>85</marks>
</student>
```

JavaScript cannot use the XML efficiently merely by looking at the raw text. A parser converts the XML into a document structure from which the program can retrieve name and marks.

3.5.16 Need for XML Parsers

An XML document is fundamentally text.

Example:

```
<product>
    <name>Laptop</name>
    <price>50000</price>
</product>
```

A program may need to obtain only:

Laptop

or:

50000

The parser interprets the XML structure and enables programmatic access to the required elements.

Conceptually:

XML Text

↓

XML Parser

↓

Document/Data Structure

↓

Application Program

3.5.17 Types of XML Parsers

At an introductory level, two common approaches are:

1. DOM-based parsing
2. SAX-based parsing

3.5.18 DOM Parser

Definition: A **DOM-based XML parser reads an XML document and represents it as a tree of nodes in memory according to the Document Object Model.** Elements, attributes, and text values can then be navigated and accessed through the tree structure. DOM parsing is convenient when the application needs to move freely through different parts of a document or modify its content. However, storing the entire document tree in memory may require more resources for very large XML files.

XML Example

```
<student>
    <name>Ananya</name>
    <marks>85</marks>
</student>
```

A simplified DOM representation is:

student

name

Ananya

marks

85

Scenario

A college website receives a small XML student record and needs to access the student's name and marks several times. DOM parsing is suitable because the entire document can be easily navigated.

3.5.19 SAX Parser

Definition: A SAX parser is an event-based XML parser that reads an XML document sequentially and reports events as elements, attributes, and text are encountered. Unlike a DOM parser, it does not normally construct and keep the complete document tree in memory. This makes SAX-style parsing suitable for large XML documents and sequential processing. However, accessing previously processed information is less convenient because the parser moves through the document as a stream.

Conceptual Working

For:

```
<student>
    <name>Ananya</name>
</student>
```

a SAX-style parser conceptually reports events such as:

Start student

Start name

Text: Ananya

End name

End student

Scenario

A large university database exports thousands of student records in one XML file. A streaming parser can process records one by one without keeping the complete document structure in memory.

3.5.20 DOM Parser and SAX Parser – Comparison

DOM Parser	SAX Parser
Builds a complete tree	Processes XML sequentially
Stores document in memory	Does not normally store the complete tree
Easy random access	Mainly sequential access
Easier for small and medium XML documents	Suitable for very large XML documents
Convenient for modification	Mainly suited to reading/processing streams

3.5.21 DOMParser in JavaScript

Definition: The **DOMParser interface provided by web browsers converts a string containing markup such as XML into a DOM Document object.** The resulting document can then be accessed using familiar DOM methods such as getElementsByTagName(). It provides a convenient browser-side method for processing small XML documents. DOMParser parses XML structure, but browser use of DOMParser should not be confused with full XSD schema validation.

Syntax

```
const parser = new DOMParser();
const xmlDocument =
      parser.parseFromString(
            xmlText,
            "application/xml"
      );
```

3.5.22 XML Parsing Example

```
const xmlText =
      "<student>" +
      "<name>Ananya</name>" +
      "<marks>85</marks>" +
      "</student>";

const parser =
      new DOMParser();

const xmlDocument =
      parser.parseFromString(
            xmlText,
            "application/xml"
      );
```

```
const studentName =
        xmlDocument
                .getElementsByTagName("name")[0]
                .textContent;

console.log(studentName);
```

Output:

Ananya

Scenario

A student portal receives XML data containing a student's details. JavaScript parses the XML and extracts the required name before displaying it on the webpage.

3.5.23 Complete Browser Program Using an XML Parser

```
<!DOCTYPE html>
<html>
<head>
        <title>XML Parser Example</title>
</head>
<body>
<h2>Student Information</h2>
<p id="result"></p>
```

```
<script>

const xmlText =

    "<student>" +

        "<name>Ananya</name>" +

        "<department>CSE</department>" +

        "<marks>85</marks>" +

    "</student>";

const parser =

    new DOMParser();

const xmlDocument =

    parser.parseFromString(

        xmlText,

        "application/xml"

    );

const name =

    xmlDocument

        .getElementsByTagName("name")[0]

        .textContent;

const department =

    xmlDocument

        .getElementsByTagName("department")[0]

        .textContent;
```

```
const marks =
        xmlDocument
                .getElementsByTagName("marks")[0]
                .textContent;
document.getElementById("result").innerHTML =
        "Name: " + name +
        "<br>Department: " + department +
        "<br>Marks: " + marks;
</script>
</body>
</html>
```

3.5.25 XML Schema and XML Parser – Difference

XML Schema	XML Parser
Defines rules for XML documents	Reads and interprets XML documents
Specifies structure and data types	Converts XML into a processable form
Commonly stored in .xsd files	Implemented as software/API
Can define required elements	Finds and reads elements
Used for validation rules	Used for parsing and processing

A simple way to remember the difference is:

Schema → Defines what XML should look like

Parser → Reads the XML document

3.5.26 Scenario

Consider a university examination system.

The university defines an XSD stating that every result must contain:

Student ID

Name

Subject

Marks

and marks must be numeric.

A college then sends:

```
<result id="101">
    <name>Ananya</name>
    <subject>Web Programming</subject>
    <marks>85</marks>
</result>
```

The XML Schema can be used by suitable validation software to check whether the structure follows the university rules.

After the XML is accepted, an XML parser reads the document and allows the application to retrieve:

Ananya

Web Programming

85

Therefore:

XSD

↓

Defines Rules

XML

↓

Contains Data

Parser

↓

Reads Data

Application

↓

Uses Data

3.5.27 Scenario-Based Questions

Question 1: A university wants every student XML record to contain name, department, and numeric marks. Which technology defines these rules?

Answer: XML Schema (XSD).

Question 2: Which XSD type is suitable for a student's name?

Answer: xs:string.

Question 3: Which XSD type is suitable for examination marks stored as whole numbers?

Answer: xs:integer.

Question 4: A student element contains name, department, and marks as child elements. Should it normally use a simple or complex type?

Answer: A complex type.

Question 5: A student ID attribute must always appear. Which setting can be used?

Answer:

use="required"

Question 6: A program needs to read XML and access its elements. Which software component is required?

Answer: An XML parser.

Question 7: Which parser approach creates a complete document tree in memory?

Answer: DOM parsing.

Question 8: Which parser approach is more suitable for sequential processing of very large XML files?

Answer: SAX-style parsing.

Question 9: Which browser interface can JavaScript use to parse an XML string into a DOM document?

Answer: DOMParser.

3.5.28 Important Questions

1. What is XML Schema?

XML Schema is a formal language used to describe the permitted structure, content, and data constraints of XML documents. It can define elements, attributes, data types, occurrence rules, and relationships among elements.

2. What is XSD?

XSD stands for XML Schema Definition and commonly refers to the W3C XML Schema language and schema documents used to define XML rules.

3. What is the difference between a simple type and a complex type?

A simple type represents a value without child elements, while a complex type can contain child elements, attributes, or other structured content.

4. What is a valid XML document?

A valid XML document is a well-formed document that also satisfies the rules defined by its associated schema or validation grammar.

5. What is an XML parser?

An XML parser is software that reads XML markup, checks its structural syntax, and makes its content available to an application.

6. What is a DOM parser?

A DOM parser creates a complete tree representation of an XML document in memory and allows programs to navigate its nodes.

7. What is a SAX parser?

A SAX parser processes an XML document sequentially and reports parsing events without normally storing the entire document tree.

8. What is DOMParser?

DOMParser is a browser interface that converts an XML or other supported markup string into a DOM Document.

3.5.29 Summary

XML Schema defines the rules that XML documents are expected to follow. It can specify elements, attributes, data types, ordering, and occurrence constraints. Simple types represent individual values, while complex types describe structured elements containing child elements or attributes.

A well-formed XML document follows XML syntax rules, while a valid XML document additionally satisfies the applicable schema rules.

An XML parser reads XML and converts it into a form that application programs can process. DOM-based parsers create a complete document tree, while SAX-style parsers process XML sequentially. In browser JavaScript, DOMParser provides a convenient method for converting XML text into a DOM document.

3.6 AJAX: Introduction and Working Principle

3.6.1 Introduction

Traditional webpages commonly follow a complete request-response cycle. When a user submits a form or requests new information, the browser sends a request to the server and may reload the entire webpage to display the response. Repeated page reloads can make an application feel slower and interrupt the user's interaction with the page.

AJAX provides a different approach. JavaScript can communicate with a server in the background, receive the required information, and update only the necessary part of the webpage. This allows websites to provide a faster and more interactive user experience.

For example, when a student enters a registration number on a result portal, the result can be displayed below the search box without loading an entirely new page.

3.6.2 AJAX

Definition: AJAX stands for Asynchronous JavaScript and XML and refers to a web-development technique in which JavaScript communicates with a server asynchronously and updates selected parts of a webpage without requiring a complete page reload. AJAX is not a separate programming language or software product; it is an approach that combines existing web technologies. JavaScript controls the communication and webpage update, while data may be exchanged using XML, JSON, text, or other formats. Although the name contains XML, modern AJAX applications frequently use JSON.

Traditional approach:

Browser → Request → Server → New Complete Page

AJAX approach:

Browser → Background Request → Server → Data → Update Required Section

Scenario

Consider an e-commerce website. A customer selects a different product category.

Without AJAX, the complete webpage may reload to display the selected products.

With AJAX, JavaScript can request only the required product information and update the product section while the header, menu, and other webpage content remain unchanged.

3.6.3 Brief History of AJAX

The technologies used for AJAX existed before the term itself became popular. Browser-based asynchronous communication became practical through mechanisms that eventually developed into the XMLHttpRequest interface.

In **2005**, Jesse James Garrett popularized the term **AJAX** in the article *Ajax: A New Approach to Web Applications*. He used the term to describe web applications that communicate with servers asynchronously and update their interfaces dynamically instead of repeatedly loading complete webpages.

Early AJAX applications commonly exchanged XML data. Later, **JSON** became very popular because it is compact and convenient to process in JavaScript. Modern applications may use either XMLHttpRequest or the newer Fetch API to perform AJAX-style communication.

3.6.4 Need for AJAX

Definition: The need for AJAX arises when a web application must communicate with a server and update information without interrupting the user by reloading the entire webpage. AJAX reduces unnecessary transfer and rendering of unchanged page content by requesting only the information required for a particular operation. It supports interactive interfaces in which users can continue working while background communication takes place. This makes AJAX particularly useful for search, validation, dashboards, shopping carts, and other dynamic web applications.

Traditional Webpage Problem

Suppose a college portal contains:

Student Name
Register Number
Department
Result Section
Navigation Menu
College Logo

If the student searches for a result and the complete page reloads, all the unchanged information is loaded again.

With AJAX, only:

Result Section

needs to be updated.

Scenario

A student enters:

Register Number: 23CSE101

and clicks **View Result**.

AJAX can request the result in the background and update:

Result: PASS

without reloading the entire college portal.

3.6.5 Meaning of "Asynchronous" in AJAX

Definition: Asynchronous communication means that a web application can initiate a request without requiring the entire program or user interface to remain blocked until the response is received. JavaScript can continue handling other activities while the browser waits for the server response. When the response becomes available, the appropriate JavaScript code processes it and updates the webpage. This behaviour helps web applications remain responsive during network operations.

Example

Suppose a server requires two seconds to return product information.

During those two seconds, the user may still:

scroll the page,
read existing information, or
click another interface element.

The webpage does not have to remain completely inactive.

Scenario

A food-ordering website requests restaurant information from the server. While the request is being processed, the customer can continue viewing the current page.

3.6.6 Technologies Used in AJAX

AJAX is a technique that works through several web technologies.

HTML

HTML provides the structure of the webpage.

Example:

```
<p id="result"></p>
```

The paragraph can later display information received from a server.

CSS

CSS controls the presentation of the webpage and the dynamically updated content.

JavaScript

JavaScript sends requests, receives responses, processes data, and updates webpage elements.

DOM

The Document Object Model allows JavaScript to locate and modify specific HTML elements.

Example:

```
document.getElementById("result").textContent =
        "Result: Pass";
```

Browser Communication API

JavaScript can communicate with a server using technologies such as:

XMLHttpRequest

or:

Fetch API

Data Format

The server may return information as:

XML

JSON

Text

HTML

Modern web applications frequently use JSON.

3.6.7 AJAX Working Principle

The fundamental AJAX process contains the following stages.

Step 1 – User Performs an Action

The process normally begins with a user action such as clicking a button, selecting an option, or entering search information.

Example:

A student clicks **View Result**.

Step 2 – JavaScript Detects the Action

JavaScript responds to the event.

Conceptually:

button click

↓

JavaScript function

Step 3 – JavaScript Sends a Request

JavaScript uses a browser communication API such as XMLHttpRequest or fetch() to send an HTTP request to the server.

Step 4 – Server Processes the Request

The server receives the request and performs the required operation.

For example, it may:

search for a student result,
find product information, or
check whether a username is available.

Step 5 – Server Sends a Response

The server returns the requested information.

The response may contain XML, JSON, text, or another suitable format.

Step 6 – JavaScript Receives the Response

JavaScript processes the returned data.

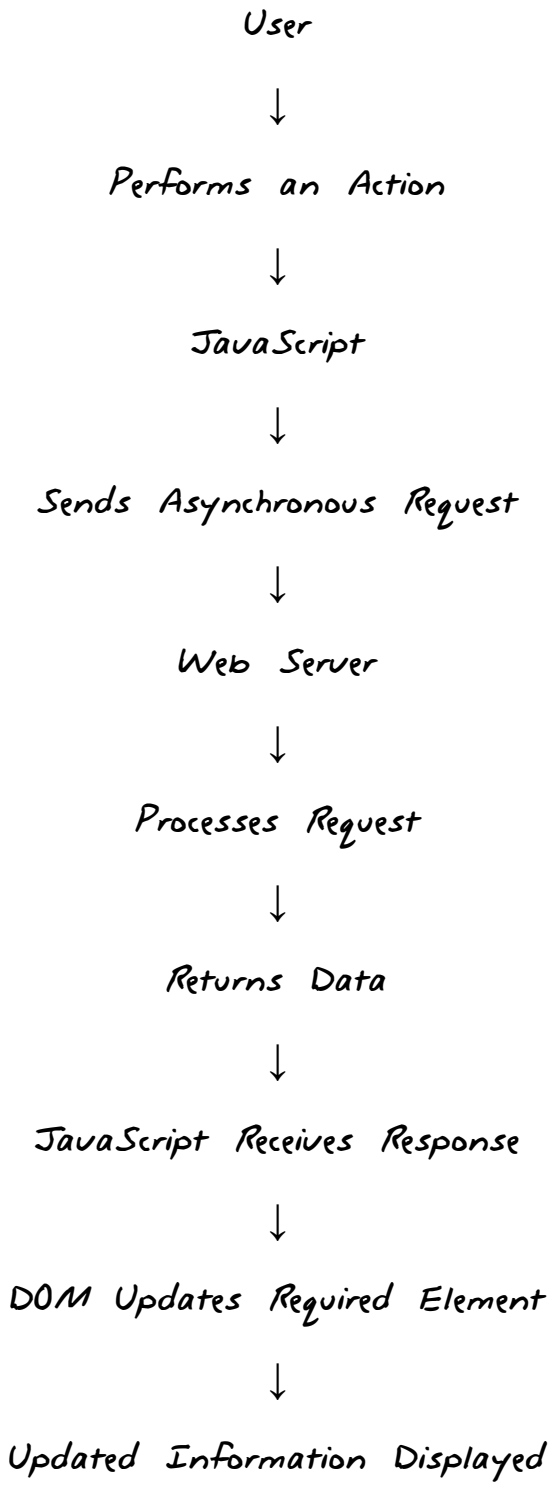
Step 7 – DOM Updates the Webpage

JavaScript updates only the required HTML element.

The complete webpage does not need to reload.

3.6.8 AJAX Working Principle - Flow

A simple AJAX flow can be represented as:



3.6.9 Scenario – College Result Portal

Consider a college result website.

The student enters:

Register Number: 23CSE101

and clicks:

View Result

The working principle is:

Student enters Register Number

↓

Clicks View Result

↓

JavaScript reads Register Number

↓

Request sent to server

↓

Server searches student record

↓

Server returns result data

↓

JavaScript receives data

↓

Result section is updated

The webpage may display:

Name: Ananya

Department: CSE

Result: PASS

Only the result area changes. The rest of the webpage remains as it is.

This is the central idea of AJAX.

3.6.10 Traditional Web Application and AJAX Application

Traditional Web Application

User Action

↓

Request

↓

Server

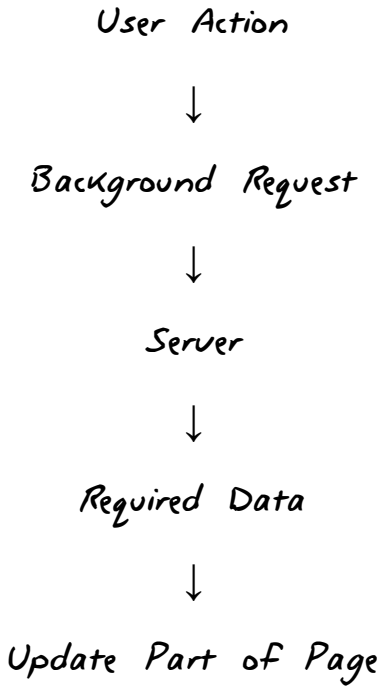
↓

Complete New Page

↓

Browser Reload

AJAX-Based Application



Comparison

Traditional Approach	AJAX Approach
Often reloads the complete page	Updates selected content
User interaction may be interrupted	Page can remain interactive
Unchanged content may be transferred again	Only required information can be requested
Simpler for basic page navigation	Suitable for dynamic applications
Each operation may lead to a new page	Many operations can occur within the same page

3.6.11 Does AJAX Always Use XML?

No.

The name AJAX contains the word **XML** because XML was commonly used for data exchange when the term became popular.

For example:

```
<student>
    <name>Ananya</name>
    <result>Pass</result>
</student>
```

However, AJAX can also use JSON:

```
{
    "name": "Ananya",
    "result": "Pass"
}
```

Modern web applications frequently prefer JSON because it maps naturally to JavaScript data structures and is generally concise.

Therefore:

AJAX does not mean that XML is compulsory.

3.6.12 Simple AJAX-Style Example Using Fetch

The Fetch API was introduced earlier in this unit and can be used to demonstrate AJAX-style communication.

```
<!DOCTYPE html>
<html>
```

```
<head>
        <title>AJAX Example</title>
</head>
<body>
<h2>User Information</h2>
<button onclick="loadUser()">
        Load User
</button>
<p id="result">
        User information will appear here.
</p>
<script>
async function loadUser() {
        const response =
                await fetch(
                        "https://jsonplaceholder.typicode.com/users/1"
                );
  const data =
                await response.json();
document.getElementById("result").textContent =
                "Name: " + data.name;
}
```

```
</script>

</body>

</html>
```

3.6.14 Advantages of AJAX

AJAX can improve the behaviour of interactive web applications because it allows background communication and selective webpage updates.

Important advantages include:

- Reduced complete-page reloads.
- Faster perceived user interaction.
- Dynamic updating of webpage content.
- Background communication with servers.
- Reduced transfer of unchanged webpage content in suitable applications.
- Improved user experience for interactive operations.

Scenario

When a customer changes the quantity of an item in a shopping cart, AJAX can update the cart total without reloading the entire shopping page.

3.6.15 Applications of AJAX

AJAX is commonly used in:

- Search suggestions.
- Shopping-cart updates.
- Username availability checking.
- Student-result portals.

- Online forms.
- Live dashboards.
- Product filters.
- Notifications.
- Chat interfaces.

Scenario

When a user starts typing:

web prog...

inside a search box, a website may retrieve and display suggestions such as:

Web Programming

Web Programming Tutorial

Web Programming Books

without reloading the complete page.

3.6.16 Important Clarification

AJAX is:

A web-development technique

AJAX is not:

A programming language

A database

A web server

A separate software package

JavaScript performs the client-side programming, while APIs such as XMLHttpRequest or Fetch provide network communication.

3.6.17 Scenario-Based Questions

Question 1: A shopping website changes the cart total without refreshing the entire webpage. Which technique is being used?

Answer: AJAX.

Question 2: A college portal retrieves a student's result from a server while keeping the existing webpage visible. What AJAX characteristic makes this possible?

Answer: Asynchronous communication.

Question 3: Which programming language normally controls AJAX operations in the browser?

Answer: JavaScript.

Question 4: Which browser technologies can be used to send AJAX-style requests?

Answer: XMLHttpRequest or the Fetch API.

Question 5: Does AJAX require XML for every response?

Answer: No. XML, JSON, text, or other suitable response formats may be used.

Question 6: A webpage receives new data and changes only a <p> element. Which web model allows JavaScript to update that element?

Answer: The Document Object Model (DOM).

Question 7: A search website displays suggestions while the user types without reloading the page. Which web-development technique is suitable?

Answer: AJAX.

3.6.18 Important Questions

1. What is AJAX?

AJAX stands for Asynchronous JavaScript and XML. It is a web-development technique in which JavaScript communicates with a server asynchronously and dynamically updates selected parts of a webpage without requiring a complete page reload.

2. Is AJAX a programming language?

No. AJAX is a technique that combines technologies such as JavaScript, DOM operations, HTTP communication, and structured response data.

3. Why is AJAX required?

AJAX is required when a web application needs to exchange information with a server while maintaining a responsive webpage and updating only the required content.

4. What does asynchronous mean in AJAX?

Asynchronous means that a request can be initiated without requiring the entire webpage to remain blocked while waiting for the response.

5. What are the main stages in AJAX communication?

A user action occurs, JavaScript sends a request, the server processes it, a response is returned, JavaScript processes the response, and the DOM updates the required webpage content.

6. Which data formats can AJAX use?

AJAX can work with XML, JSON, text, HTML, and other suitable response formats.

7. What is the role of the DOM in AJAX?

The DOM enables JavaScript to locate and update selected HTML elements after new information is received.

8. What is the difference between a traditional request and AJAX?

A traditional interaction may cause the entire webpage to reload, whereas AJAX can request data in the background and update only the required part of the page.

3.6.19 Summary

AJAX stands for **Asynchronous JavaScript and XML** and is a technique for creating dynamic and interactive web applications. It allows JavaScript to communicate with a server without requiring a complete webpage reload for every operation.

The AJAX working principle begins with a user action. JavaScript sends an asynchronous request, the server processes it and returns data, JavaScript receives the response, and the DOM updates only the required part of the webpage.

Although XML was historically associated with AJAX, XML is not compulsory. Modern AJAX applications frequently use JSON and may perform communication using either XMLHttpRequest or the Fetch API.

3.7 XML HttpRequest

3.7.1 Introduction

In the previous topic, AJAX was introduced as a technique that allows a webpage to communicate with a server and update selected content without reloading the entire page. One of the traditional browser interfaces used to perform this communication is **XMLHttpRequest**.

For example, a college result portal may send a student's registration number to a server, receive the corresponding result, and display it in the existing webpage. XMLHttpRequest provides JavaScript with the methods and properties required to create the request, send it, monitor its progress, receive the response, and process the returned information.

Although modern applications frequently use the Fetch API, understanding XMLHttpRequest is important because it forms an important part of the development and working principle of AJAX.

3.7.2 XML HttpRequest

Definition: XMLHttpRequest, commonly abbreviated as XHR, is a browser-provided programming interface that allows JavaScript to exchange data with a web server through HTTP requests. It can perform communication asynchronously, allowing the webpage to remain available while the browser waits for the server response. The received response can contain text, XML, JSON, or other supported data formats. XMLHttpRequest is one of the fundamental technologies traditionally associated with AJAX-based web applications.

The working process is:

Browser

→ Create XMLHttpRequest object

→ Prepare request

→ Send request

→ Server processes request

→ Receive response

→ JavaScript processes response

→ Update webpage

Scenario

A library website contains a **Check Availability** button. When the user clicks the button, JavaScript can use XMLHttpRequest to request the current availability of the selected book and display the result without opening a new webpage.

3.7.3 Need for XMLHttpRequest

Before asynchronous browser communication became common, obtaining new information from a server usually involved loading another webpage or reloading the existing page.

XMLHttpRequest allows JavaScript to request only the required information. This makes it possible to develop features such as live search, availability checking, dynamic result display, and shopping-cart updates.

Scenario

Suppose an e-commerce page already displays:

Product Name

Product Image

Price

Description

The customer clicks:

Check Stock

Instead of refreshing the complete product page, XMLHttpRequest can request only the latest stock information and display:

12 items available

3.7.4 Creating an XMLHttpRequest Object

Definition: An XMLHttpRequest object is an instance of the browser's XMLHttpRequest interface and represents one HTTP request-response operation. The object provides methods for preparing and sending the request and properties for examining its progress and response. A new object is normally created for a request using the new keyword. The program then configures this object before sending it to the server.

Syntax

```
const xhr = new XMLHttpRequest();
```

Here:

XMLHttpRequest is the browser interface.

new creates an object.

xhr is the variable referring to the XMLHttpRequest object.

Scenario

A college result portal creates an XHR object when it needs to request a student's result from the server.

```
const xhr = new XMLHttpRequest();
```

3.7.5 The open() Method

Definition: The open() method initializes an XMLHttpRequest by specifying the HTTP method, the requested resource, and whether the request should operate asynchronously. Calling open() prepares the request but does not actually send it to the server. Common HTTP methods include GET for requesting information and POST for sending information. In normal AJAX applications, asynchronous operation is preferred.

Syntax

```
xhr.open(method, url, async);
```

Example

```
xhr.open(
        "GET",
        "student.txt",
        true
);
```

Here:

GET specifies the HTTP request method.

student.txt is the requested resource.

true indicates asynchronous communication.

Scenario

A college webpage needs to obtain a notice stored on the server. The program can prepare a GET request for that resource using open().

3.7.6 GET Request

Definition: GET is an HTTP request method primarily used to retrieve a representation of a resource from a server. In an XMLHttpRequest operation, GET is commonly used when the browser needs information without submitting a request body containing new application data. The requested resource is specified through its URL. GET requests are widely used for retrieving student information, products, notices, and other web resources.

Example

```
xhr.open(
        "GET",
        "notice.txt",
        true
);
```

Scenario

A university website wants to retrieve the latest examination notice from the server. Since the webpage is requesting information, a GET request is appropriate.

3.7.7 The send() Method

Definition: The send() method sends a previously prepared XMLHttpRequest to the server. It is normally called after the request has been configured using open(). For a simple GET request, send() is usually called without a request body. After the request is sent, the browser begins receiving information about the progress and eventual response.

Syntax

```
xhr.send();
```

Example

```
const xhr = new XMLHttpRequest();

xhr.open(
        "GET",
        "notice.txt",
        true
);


xhr.send();
```

Scenario

After a library webpage prepares a request for current book availability, send() actually starts the communication with the server.

3.7.8 The readyState Property

Definition: The readyState property indicates the current state of an XMLHttpRequest during its request-response lifecycle. Its value changes as the request is created, opened, sent, and completed. JavaScript can examine this property to determine whether the server response is ready for processing. The final state, 4, indicates that the request operation has completed.

XMLHttpRequest States

Value	State	Meaning
0	UNSENT	Request object created, but open() not yet called
1	OPENED	open() has been called
2	HEADERS_RECEIVED	Response headers have been received
3	LOADING	Response data is being received
4	DONE	Request operation has completed

xhr.readyState === 4

because it indicates completion.

Scenario

A student portal should not display the requested result while only part of the response has arrived. It can wait until readyState becomes 4 before processing the completed response.

3.7.9 The onreadystatechange Event Handler

Definition: onreadystatechange is an event-handler property that allows JavaScript to execute a function whenever the readyState value of an

XMLHttpRequest changes. It was traditionally used to monitor the different stages of an AJAX request. The handler can examine readyState and the HTTP status before processing the response. This makes it possible to react when a request has completed successfully.

Syntax

```
xhr.onreadystatechange = function () {
    // Check request state
};
```

Example

```
xhr.onreadystatechange = function () {

    if (xhr.readyState === 4) {
        console.log("Request completed");
    }

};
```

Scenario

A hospital appointment webpage requests doctor availability. The function assigned to onreadystatechange can wait until the server response has completely arrived before displaying the available time slots.

3.7.10 The status Property

Definition: The status property contains the HTTP status code associated with the XMLHttpRequest response when such a response is available. HTTP status codes indicate whether the server successfully processed the request or whether a problem occurred. A status of 200 normally represents a successful HTTP response. Programs commonly examine both readyState and status before using the returned data.

Common Status Codes

Status	Meaning
200	Request successful
404	Requested resource not found
500	Server encountered an error

Common Check

```
if (
      xhr.readyState === 4 &&
      xhr.status === 200
) {
      // Process response
}
```

Scenario

A library portal requests books.txt. If the server returns status 200, the data can be displayed. If the resource does not exist and the server returns 404, the website can show an error message instead.

3.7.11 The responseText Property

Definition: The responseText property provides the response body of an XMLHttpRequest as text when the response is being handled as textual data. It allows JavaScript to read information returned by the server and use it in the webpage. The returned text may represent ordinary text or a textual data format such as XML or JSON. The application can subsequently process the text according to its expected format.

Example

```
console.log(xhr.responseText);
```

If the server returns:

Admissions are open.

then responseText contains that text.

Scenario

A college website requests the latest admission notice. After the request succeeds, responseText can contain:

Admissions are open until 30 August.

JavaScript can display this message in the webpage.

3.7.12 Basic XMLHttpRequest Sequence

The basic sequence can be remembered as:

1. Create
2. Open
3. Define response handling
4. Send

5. Receive response

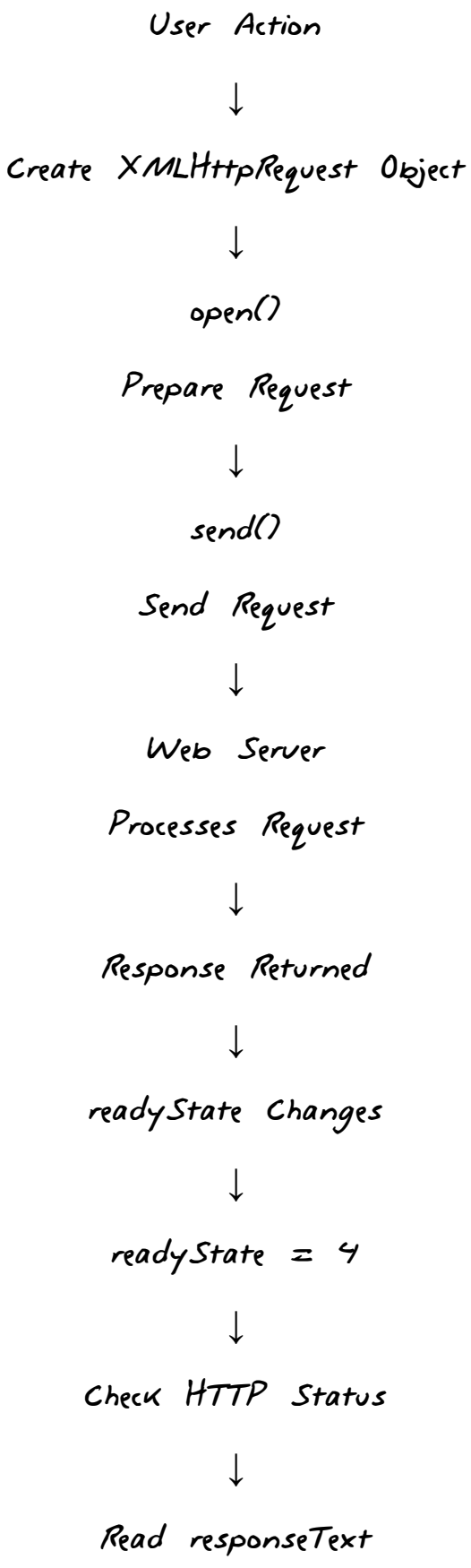
6. Update webpage

In JavaScript:

```
const xhr = new XMLHttpRequest();
xhr.open(
        "GET",
        "notice.txt",
        true
);
xhr.onreadystatechange = function () {
        if (
                xhr.readyState === 4 &&
                xhr.status === 200
        ) {
                console.log(xhr.responseText);
        }
};
xhr.send();
```

3.7.13 Working Principle

The complete process can be represented as:



↓

Update DOM

3.7.14 Real Website Scenario – College Notice Portal

Consider a college website containing a button:

Load Latest Notice

When the student clicks the button:

1. JavaScript creates an XMLHttpRequest object.
2. open() prepares the request.
3. send() sends the request.
4. The server returns the latest notice.
5. JavaScript checks whether the request completed successfully.
6. responseText reads the returned notice.
7. The DOM displays the notice without reloading the webpage.

3.7.15 Program

The following example requests data from a public demonstration web service. At this stage, the returned information is displayed as text. JSON processing will be studied separately in Topic 3.8.

```
<!DOCTYPE html>

<html>

<head>

    <title>XMLHttpRequest Example</title>

</head>
```

```
<body>
<h2>Online Data Request</h2>
<button onclick="loadData()">
    Load Data
</button>
<p id="statusMessage"></p>
<pre id="result"></pre>
<script>
function loadData() {
    const xhr =
        new XMLHttpRequest();
    xhr.open(
        "GET",
        "https://jsonplaceholder.typicode.com/posts/1",
        true
    );
document.getElementById(
        "statusMessage"
    ).textContent =
        "Loading data...";
xhr.onreadystatechange = function () {
```

```
        if (xhr.readyState === 4) {
            if (xhr.status === 200) {
                document.getElementById(
                    "statusMessage"
                ).textContent =
                    "Data received successfully.";
                document.getElementById(
                    "result"
                ).textContent =
                    xhr.responseText;
            }
            else {
                document.getElementById(
                    "statusMessage"
                ).textContent =
                    "Unable to load data.";     } } };
            xhr.send();   }
</script>
</body>
</html>
```

3.7.16 Modern load Event

Another way to process a completed XMLHttpRequest is the load event.

Definition: The load event of an XMLHttpRequest occurs when the request has completed successfully at the network-transfer level and a response has been received. It provides a simpler event-oriented alternative to manually checking every readyState change when only completion is important. The HTTP status should still be checked because receiving a response does not necessarily mean that the server returned a successful HTTP status.

Example

```
xhr.onload = function () {
    if (xhr.status === 200) {
        console.log(xhr.responseText);
} };
```

Scenario

A product webpage only needs to act after the server response has arrived. The load handler can then process the completed response.

For this syllabus, understanding readyState and onreadystatechange remains important because they explain the traditional working of XMLHttpRequest and AJAX.

3.7.17 XMLHttpRequest and Fetch API

Both XMLHttpRequest and Fetch can communicate with web servers.

XMLHttpRequest	Fetch API
Older browser communication interface	Newer Promise-based interface
Uses an XHR object	Uses fetch()
Traditionally uses events and readyState	Naturally works with Promises
Supports responseText and other response forms	Uses a Response object
Important in traditional AJAX	Common in modern JavaScript

XMLHttpRequest

```
const xhr =
        new XMLHttpRequest();
xhr.open(
        "GET",
        "data.txt",
        True   );


xhr.send();
```

Fetch

```
fetch("data.txt");
```

Both can perform AJAX-style communication, but their programming styles are different.

3.7.18 Important Points

new XMLHttpRequest() creates the request object.

open() prepares the request.

send() sends the request.

readyState tells the current request stage.

status provides the HTTP response status.

responseText contains a textual response.

onreadystatechange allows JavaScript to react when the request state changes.

A simple memory sequence is:

Create → Open → Monitor → Send → Check → Read → Display

3.7.19 Scenario-Based Questions

Question 1: A college webpage needs to request the latest examination notice without refreshing the complete page. Which browser interface can be used?

Answer: XMLHttpRequest.

Question 2: Which statement creates an XMLHttpRequest object?

Answer:

```
const xhr =
      new XMLHttpRequest();
```

Question 3: Which method prepares an XMLHttpRequest by specifying the HTTP method and URL?

Answer: open().

Question 4: Which method actually sends the prepared request?

Answer: send().

Question 5: A program needs to know whether an XMLHttpRequest has completely finished. Which property should it examine?

Answer: readyState.

Question 6: Which readyState value represents a completed request?

Answer: 4.

Question 7: Which HTTP status code normally indicates a successful request?

Answer: 200.

Question 8: A server returns a textual notice. Which XMLHttpRequest property can retrieve that text?

Answer: responseText.

3.7.20 Important Questions

1. What is XMLHttpRequest?

XMLHttpRequest is a browser-provided interface that allows JavaScript to send HTTP requests to a server and receive responses. It is commonly associated with traditional AJAX communication and can operate asynchronously.

2. Why is XMLHttpRequest used?

It allows webpages to request server information and dynamically update selected content without requiring a complete webpage reload.

3. What is the purpose of open()?

The open() method initializes a request by specifying information such as the HTTP method, resource URL, and asynchronous behaviour.

4. What is the purpose of send()?

The send() method sends the prepared XMLHttpRequest to the server.

5. What is readyState?

readyState indicates the current stage of an XMLHttpRequest. The value 4 indicates that the request operation has completed.

6. What is the status property?

The status property provides the HTTP status code of the response, such as 200 for a successful HTTP response.

7. What is responseText?

responseText provides the response body as text when the response is handled as textual data.

8. What is onreadystatechange?

onreadystatechange is an event-handler property that executes a function whenever the request's readyState changes.

9. What is the difference between XMLHttpRequest and Fetch?

XMLHttpRequest uses an object-and-event-based programming model, while Fetch provides a newer Promise-based interface for network requests.

3.7.22 Summary

XMLHttpRequest, commonly called **XHR**, is a browser interface used by JavaScript to communicate with web servers. It played a central role in traditional AJAX applications by allowing asynchronous requests and selective webpage updates.

An XMLHttpRequest object is created using new XMLHttpRequest(). The open() method prepares the request and send() sends it. The readyState property indicates request progress, while the status property provides the HTTP response status. When a response is received as text, it can be accessed through responseText.

The fundamental sequence is:

Create → Open → Send → Receive → Check → Process → Update

3.8 JSON Response Handling

3.8.1 Introduction

In the previous topics, we learned that JavaScript can communicate with a server using the **Fetch API** or **XMLHttpRequest**. After a request is sent, the server returns a response containing the requested information. The client-side JavaScript must understand this response before the information can be displayed or processed.

Modern web applications commonly exchange information in **JSON (JavaScript Object Notation)** format. JSON is compact, readable, and easy for JavaScript to process. For example, a college server may return a student's name, department, and marks as JSON, and JavaScript can extract these values and display them on the webpage.

3.8.2 JSON

Definition: JSON, or JavaScript Object Notation, is a lightweight text-based data-interchange format used to represent and exchange structured information between applications. JSON represents information using name-value pairs, arrays, and simple data values such as strings, numbers, Boolean values, and null. Although its syntax is derived from JavaScript object notation, JSON is a language-independent data format and can be processed by many programming languages. It is widely used for exchanging information between web browsers, servers, and web APIs.

Example

```
{
    "name": "Ananya",
    "department": "CSE",
```

```
    "marks": 85
}
```

This JSON document contains three pieces of information:

name → Ananya

department → CSE

marks → 85

Scenario

A college result server may return:

```
{
    "rollNo": 101,
    "name": "Ananya",
    "result": "Pass"
}
```

JavaScript can read these values and display the student's result without reloading the complete webpage.

3.8.3 Need for JSON

Web applications frequently exchange information between the browser and the server. A common and predictable data format is required so that both sides can understand the information correctly.

For example, an e-commerce server may need to send:

Product Name

Price

Stock

Rating

Instead of sending unrelated pieces of text, the server can organize them as:

```
{
    "product": "Laptop",
    "price": 50000,
    "stock": 12,
    "rating": 4.5
}
```

JSON is especially suitable for modern web applications because it is relatively compact and maps naturally to JavaScript objects and arrays.

Scenario

When a customer selects a product, the server can return its information as JSON. JavaScript can then display only the product section of the page.

3.8.4 JSON Syntax

JSON commonly represents information using **objects** and **arrays**.

A JSON object is enclosed within curly braces:

```
{
    "name": "Rahul",
    "age": 18
}
```

A JSON array is enclosed within square brackets:

```
[
    "HTML",
    "CSS",
    "JavaScript"
]
```

In a JSON object, a property is represented as:

"name" : value

Property names must be written as JSON strings using double quotation marks.

3.8.5 JSON Data Types

JSON supports several basic types of values.

String

A string stores textual information.

```
{
    "name": "Ananya"
}
```

Number

A number stores numeric information.

```
{
```

```
    "marks": 85
}
```

Boolean

A Boolean value is either true or false.

```
{
    "isPassed": true
}
```

Null

null represents the intentional absence of a value.

```
{
    "middleName": null
}
```

Array

An array stores an ordered collection of values.

```
{
    "subjects": [
        "HTML",
        "CSS",
        "JavaScript"
    ] }
```

Object

An object stores related information using name-value pairs.

```
{
    "student": {
        "name": "Ananya",
        "department": "CSE"
    }  }
```

Scenario

A student portal may receive one JSON response containing text, numbers, Boolean values, arrays, and nested objects representing the complete student record.

3.8.6 JSON Response

Definition: A JSON response is response data sent by a server in JSON format for processing by a client application. The response may contain one object, a collection of objects, arrays, or nested structured information. JavaScript must obtain the response body and convert or interpret it as JavaScript data before accessing its individual values. JSON responses are widely used by web APIs because they provide a structured and convenient method of exchanging application data.

Example

A server may return:

```
{
    "name": "Ananya",
    "department": "CSE",
```

```
    "marks": 85
}
```

JavaScript can then access:

data.name

data.department

data.marks

Scenario

A result portal sends a registration number to the server. The server returns the student's result as JSON, and JavaScript displays it in the result section.

3.8.7 JSON.parse()

Definition: JSON.parse() is a JavaScript method that converts a valid JSON-formatted text string into the corresponding JavaScript value or object. It is commonly required when JSON has been received as ordinary text, such as through the responseText property of an XMLHttpRequest. After parsing, JavaScript can access the resulting object's properties using normal object notation. If the supplied text does not contain valid JSON syntax, JSON.parse() throws an error.

Syntax

```
const data = JSON.parse(jsonText);
```

Example

```
const jsonText =
```

```
'{"name":"Ananya","marks":85}';

const student =
    JSON.parse(jsonText);

console.log(student.name);
console.log(student.marks);
```

Output:

Ananya

85

Scenario

An AJAX request receives:

{"product":"Laptop","price":50000}

as text. JSON.parse() converts the text into a JavaScript object so that the website can access product and price.

3.8.8 Accessing JSON Object Properties

Definition: After JSON data has been converted into a JavaScript object, its values can be accessed using JavaScript property-access syntax. Dot notation is commonly used when the property name is known and follows normal JavaScript identifier rules. Bracket notation may also be used when required. Property access allows applications to extract only the information needed for display or further processing.

Example

```
const student = {
        name: "Ananya",
        department: "CSE",
        marks: 85
};
console.log(student.name);
console.log(student.department);
```

Output:

Ananya

CSE

Scenario

A college webpage may receive complete student information but display only:

Name

Department

Result

The required properties can be selected individually from the returned object.

3.8.9 Handling JSON Arrays

Definition: A **JSON array is an ordered collection of JSON values enclosed within square brackets.** It is commonly used when a server needs to return several related records, such as multiple students, products, books, or orders. After the JSON response is converted into JavaScript data, individual array elements can be accessed by their index or processed using loops. Array indexing begins with zero.

Example

```
[
    {
        "name": "Laptop",
        "price": 50000
    },
    {
        "name": "Mobile",
        "price": 20000
    }
]
```

After parsing:

```
console.log(products[0].name);
```

Output:

Laptop

Scenario

An e-commerce server may return ten products in one JSON array. JavaScript can use a loop to display every product on the webpage.

3.8.10 Nested JSON Objects

Definition: A nested JSON object is a JSON structure in which one object contains another object or array as one of its values. Nested structures allow related information to be grouped logically and represented hierarchically. After conversion to JavaScript data, nested values are accessed by following the corresponding property path. This structure is common in real web APIs where one entity contains several related details.

Example

```
{
    "name": "Ananya",
    "address": {
        "city": "Chennai",
        "state": "Tamil Nadu"
    }
}
```

JavaScript access:

```
console.log(data.address.city);
```

Output:

Chennai

Scenario

A student portal may receive a student's personal information together with a nested address object containing city, state, and postal code.

3.8.11 JSON Response with XMLHttpRequest

xhr.responseText

responseText normally provides the response body as text.

Suppose the server returns:

```
{
    "name": "Ananya",
    "department": "CSE"
}
```

JavaScript may process it as:

```
const data =
    JSON.parse(xhr.responseText);

console.log(data.name);
```

Working Sequence

Server sends JSON

↓

xhr.responseText

↓

JSON.parse()

↓

JavaScript Object

↓

Access Properties

↓

Display Information

Scenario

A college portal receives a student result through XMLHttpRequest. The response arrives as JSON text, JSON.parse() converts it into an object, and the DOM displays the student's details.

3.8.12 Example Using XMLHttpRequest

```
const xhr =
      new XMLHttpRequest();
xhr.open(
      "GET",
      "https://jsonplaceholder.typicode.com/users/1",
      true
);
xhr.onreadystatechange = function () {
      if (
            xhr.readyState === 4 &&
            xhr.status === 200
      ) {
```

```
            const data =
                  JSON.parse(xhr.responseText);
            console.log(data.name);
      }
};
xhr.send();
```

The important statement is:

JSON.parse(xhr.responseText);

It converts the JSON text response into JavaScript data.

3.8.13 JSON Response with Fetch API

Definition: When the Fetch API receives an HTTP response, the Response object provides methods for reading and interpreting the response body. The response.json() method reads the response body and parses it as JSON, returning a Promise that fulfills with the resulting JavaScript value. It therefore performs both response-body reading and JSON parsing. This method is commonly used with await in modern JavaScript applications.

Example

```
const response =
      await fetch("data.json");
const data =
```

```
        await response.json();

console.log(data.name);
```

Scenario

A shopping website requests product information using fetch(). The server returns JSON, and response.json() converts it into JavaScript data that can be displayed on the webpage.

3.8.14 JSON.parse() and response.json() – Difference

These two operations are related but should not be confused.

JSON.parse()	response.json()
Accepts a JSON string	Reads a Fetch Response body
Converts JSON text into a JavaScript value	Reads and parses response JSON
Synchronous parsing operation	Returns a Promise
Often used with xhr.responseText	Commonly used with Fetch API

XMLHttpRequest Style

```
const data =

        JSON.parse(xhr.responseText);
```

Fetch Style

```
const data =

        await response.json();
```

3.8.15 Error Handling During JSON Processing

Definition: JSON response handling may fail when the network request is unsuccessful, the server returns an unexpected response, or the received text does not contain valid JSON syntax. Applications should therefore check the request status and handle parsing or processing errors appropriately. In Promise-based code, try...catch can be used with async and await. Proper error handling prevents invalid data from causing an uncontrolled failure in the webpage.

Example

```
try {
        const response =
            await fetch("data.json");
    if (!response.ok) {
            throw new Error("Request failed");
     }
    const data =
            await response.json();
     console.log(data);
}
catch (error) {
    console.log("Unable to load data");
}
```

Scenario

If a college result server is unavailable, the webpage should display:

Unable to load result.

instead of leaving an empty result section.

3.8.16 JSON.stringify()

Definition: JSON.stringify() is a JavaScript method that converts a JavaScript value or object into a JSON-formatted string. It performs the reverse general operation of JSON.parse() and is useful when JavaScript data must be stored or transmitted as JSON text. Objects, arrays, strings, numbers, Boolean values, and supported nested structures can be represented in JSON form. Certain JavaScript-specific values are handled according to JSON serialization rules.

Example

```
const student = {
        name: "Ananya",
        marks: 85
};
const jsonText =
        JSON.stringify(student);
console.log(jsonText);
```

Output:

```
{"name":"Ananya","marks":85}
```

Scenario

A registration webpage creates a JavaScript object containing student details. Before transmitting the information to a server as JSON, the program may convert the object using JSON.stringify().

Difference

JSON.parse()

JSON Text → JavaScript Object

JSON.stringify()

JavaScript Object → JSON Text

3.8.17 JSON Response Handling

The following program retrieves JSON data using XMLHttpRequest, converts the response into a JavaScript object, and displays selected information.

```
<!DOCTYPE html>
<html>
<head>
    <title>JSON Response Handling</title>
</head>
<body>
<h2>User Information</h2>
```

```
<button onclick="loadUser()">
        Load User
</button>
<p id="message"></p>
<div id="result"></div>
<script>

function loadUser() {
   const xhr =
             new XMLHttpRequest();
            document.getElementById("message").textContent =
             "Loading information...";
xhr.open(
             "GET",
             "https://jsonplaceholder.typicode.com/users/1",
             true
       );
xhr.onreadystatechange = function () {
if (xhr.readyState === 4) {
if (xhr.status === 200) {
  const data =
                                        JSON.parse(xhr.responseText);
                                      document.getElementById(
```

```
                "message"
            ).textContent =
                "Information received.";
                document.getElementById(
                "result"
            ).innerHTML =
                "Name: " + data.name +
                "<br>Email: " + data.email +
                "<br>City: " +
                data.address.city; }
        else {
                document.getElementById(
                "message"
            ).textContent =
                "Unable to load information."; }
};
                xhr.send();
</script>
</body>
</html>
```

3.8.18 Scenario – Student Result Portal

Consider a college result portal.

The server returns:

```
{
    "registerNumber": "23CSE101",
    "name": "Ananya",
    "department": "CSE",
    "marks": 85,
    "result": "Pass"
}
```

The complete process is:

Student enters Register Number

↓

JavaScript sends request

↓

Server returns JSON

↓

JavaScript receives response

↓

JSON response is parsed

↓

Required properties are accessed

↓

DOM displays student result

The webpage may display:

Register Number: 23CSE101

Name: Ananya

Department: CSE

Marks: 85

Result: Pass

This is a typical example of JSON response handling in a web application.

3.8.20 XML and JSON – Brief Comparison

XML	JSON
Uses markup tags	Uses objects and arrays
Example: <name>Ananya</name>	Example: "name": "Ananya"
Supports attributes and namespaces	Uses name-value structures
Common in document and enterprise systems	Very common in web APIs
Parsed as XML	Parsed as JSON
Generally more verbose	Generally more compact

XML

```
<student>
    <name>Ananya</name>
    <marks>85</marks>
</student>
```

JSON

```
{
    "name": "Ananya",
    "marks": 85
}
```

Both can represent structured data. The appropriate choice depends on the application and the communication standard being used.

3.8.21 Scenario-Based Questions

Question 1: A web server returns product information using name-value pairs enclosed within curly braces. Which data format is being used?

Answer: JSON.

Question 2: An XMLHttpRequest receives JSON as responseText. Which JavaScript method can convert it into an object?

Answer: JSON.parse().

Question 3: A Fetch request receives a JSON response. Which method can read and parse the response body?

Answer: response.json().

Question 4: A shopping server returns several products in one response. Which JSON structure is suitable?

Answer: A JSON array.

Question 5: A student record contains an address object inside the main student object. What type of JSON structure is this?

Answer: Nested JSON.

Question 6: A JavaScript object must be converted into JSON text before transmission. Which method can be used?

Answer: JSON.stringify().

Question 7: A server returns invalid JSON. What should the application provide?

Answer: Appropriate error handling, such as try...catch.

3.8.22 Important Questions

1. What is JSON?

JSON is a lightweight text-based data-interchange format used to represent and exchange structured information. It organizes data using objects, arrays, and name-value pairs and is widely used in web applications and APIs.

2. What is JSON response handling?

JSON response handling is the process of receiving JSON-formatted information from a server, converting or interpreting it as JavaScript data, accessing the required values, and using those values in the application.

3. What is JSON.parse()?

JSON.parse() converts valid JSON-formatted text into the corresponding JavaScript value or object so that its data can be accessed programmatically.

4. What is JSON.stringify()?

JSON.stringify() converts a JavaScript value or object into JSON-formatted text.

5. How is JSON handled with XMLHttpRequest?

The response can be obtained as text through responseText and converted into JavaScript data using JSON.parse().

6. How is JSON handled with the Fetch API?

The response.json() method reads the response body and parses it as JSON, returning a Promise containing the resulting JavaScript data.

7. What is a JSON array?

A JSON array is an ordered collection of JSON values enclosed within square brackets and is commonly used to represent multiple records.

8. Why is error handling important while processing JSON?

Errors may occur because of network failures, unsuccessful HTTP responses, or invalid JSON syntax. Error handling allows the webpage to respond appropriately instead of failing unexpectedly.

3.8.23 Summary

JSON stands for **JavaScript Object Notation** and is a lightweight text format widely used for exchanging structured information between browsers and servers.

When XMLHttpRequest returns JSON as text, JSON.parse() converts the text into JavaScript data. With the Fetch API, response.json() reads and parses the response body. Once converted, properties, arrays, and nested objects can be accessed using normal JavaScript syntax.

JSON.stringify() performs the reverse operation by converting JavaScript data into JSON text.

The basic JSON response-handling process is:

Request → Server Response → JSON Data → Parse → Access Values → Update Webpage